

Java Collections Framework

Java Version: 1.2

Year: 1998

Feature: Collections Framework

UniqueClassName: CollectionsFramework

1. Definition

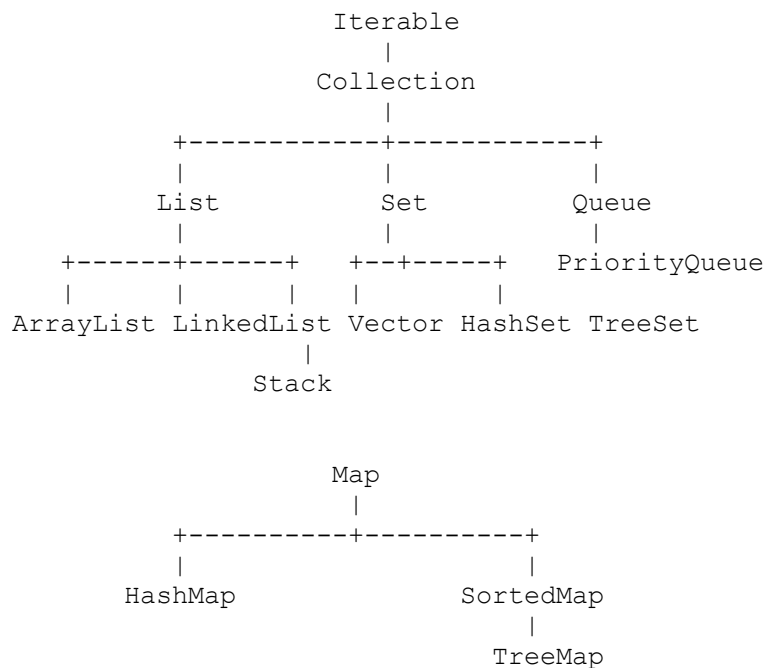
The **Collections Framework** was introduced in **Java 1.2 (1998)** to provide a standard architecture for storing, organizing, and manipulating groups of objects.

It provides:

- **Interfaces** — Collection, List, Set, Map, Queue, etc.
- **Implementations** — ArrayList, LinkedList, HashSet, TreeSet, HashMap, etc.
- **Algorithms/utility methods** — sorting, searching, reversing, shuffling, etc.
- **Iterators** — for traversing collection elements.

Note: Map is part of the Collections Framework, but it does **not** extend the Collection interface.

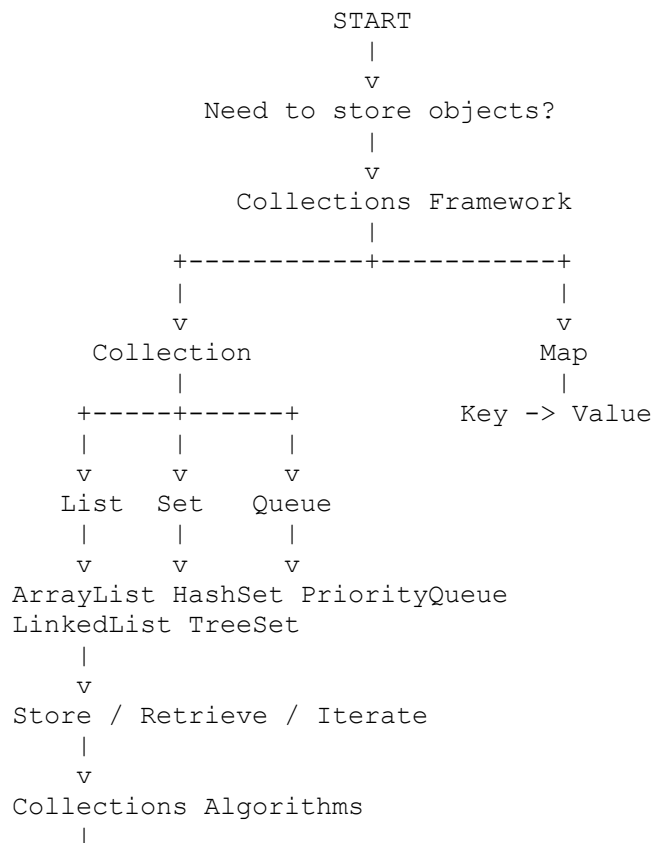
2. Main Hierarchy



Important Interfaces

Interface	Purpose	Common Implementations
Collection	Root interface for groups of objects	—
List	Ordered collection; duplicates allowed	ArrayList, LinkedList
Set	No duplicate elements	HashSet, TreeSet
Queue	Generally represents elements waiting for processing	PriorityQueue, LinkedList
Map	Key-value pairs	HashMap, TreeMap, Hashtable
SortedMap	Map whose keys are sorted	TreeMap

3. Flowchart



```
+--> sort()
+--> reverse()
+--> shuffle()
+--> binarySearch()
|
v
                                END
```

4. Java Code Example

CollectionsFramework.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class CollectionsFramework {

    public static void main(String[] args) {

        // Create a List
        List<String> languages = new ArrayList<>();

        // Add elements
        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("JavaScript");

        System.out.println("Original List:");
        System.out.println(languages);

        // Sort the collection
        Collections.sort(languages);

        System.out.println("\nSorted List:");
        System.out.println(languages);

        // Iterate through collection
        System.out.println("\nElements:");

        for (String language : languages) {
            System.out.println(language);
        }

        // Reverse the collection
        Collections.reverse(languages);

        System.out.println("\nReversed List:");
        System.out.println(languages);

        // Search for an element
        boolean exists = languages.contains("Java");
```

```

        System.out.println("\nDoes Java exist?");
        System.out.println(exists);
    }
}

```

Output

Original List:
[Java, Python, C++, JavaScript]

Sorted List:
[C++, Java, JavaScript, Python]

Elements:
C++
Java
JavaScript
Python

Reversed List:
[Python, JavaScript, Java, C++]

Does Java exist?
true

5. Example of Different Collections

```

import java.util.*;

public class CollectionsFramework {

    public static void main(String[] args) {

        // List - allows duplicates
        List<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("Java");

        System.out.println("List: " + list);

        // Set - does not allow duplicates
        Set<String> set = new HashSet<>();

        set.add("Java");
        set.add("Python");
        set.add("Java");

        System.out.println("Set: " + set);
    }
}

```

```

        // Map - key/value pairs
        Map<Integer, String> map = new HashMap<>();

        map.put(101, "Anurag");
        map.put(102, "Rahul");
        map.put(103, "Amit");

        System.out.println("Map: " + map);
    }
}

```

Conceptual Output

List: [Java, Python, Java]

Set: [Java, Python]

Map: {101=Anurag, 102=Rahul, 103=Amit}

6. Why Collections Framework Was Important

Before Java 1.2, Java had older collection-related classes such as Vector, Stack, and Hashtable. Java 1.2 introduced a more consistent architecture based on interfaces and reusable implementations.

```

Before Java 1.2
    |
    v
Different collection classes
    |
    v
Less unified design
    |
    |
Java 1.2
    |
    v
Collections Framework
    |
    +--> Interfaces
    +--> Implementations
    +--> Algorithms
    +--> Iterators
    |
    v
Reusable + Standardized Collection Handling

```

Key Point

Java 1.2 Collections Framework = Interfaces + Implementations + Algorithms + Iteration mechanisms

For example:

```
List
|
+--> ArrayList
+--> LinkedList
```

```
Set
|
+--> HashSet
+--> TreeSet
```

```
Map
|
+--> HashMap
+--> TreeMap
+--> Hashtable
```

This framework became the foundation for the modern Java collection APIs used throughout enterprise Java applications.

Java List / ArrayList

Java Version: 1.2

Year: 1998

Feature: List / ArrayList

UniqueClassName: ArrayList

1. Definition

List

List is an interface in the Java Collections Framework that represents an **ordered collection of elements**.

A List:

- Maintains **insertion order**.

- Allows **duplicate elements**.
- Allows access by **index**.
- Usually allows null elements, depending on implementation.
- Provides methods such as `add()`, `get()`, `set()`, `remove()`, and `contains()`.

ArrayList

ArrayList is a **resizable-array implementation of the List interface**.

It internally uses an array-like structure that automatically grows when additional elements are added.

```
List
|
+---- ArrayList
|
+---- LinkedList
|
+---- Vector
```

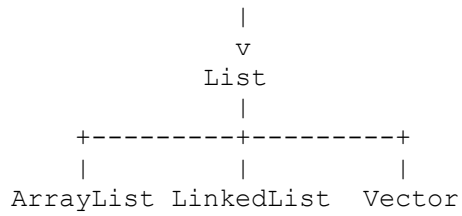
ArrayList was introduced as part of the **Java Collections Framework in Java 1.2 (1998)**.

2. ArrayList Hierarchy

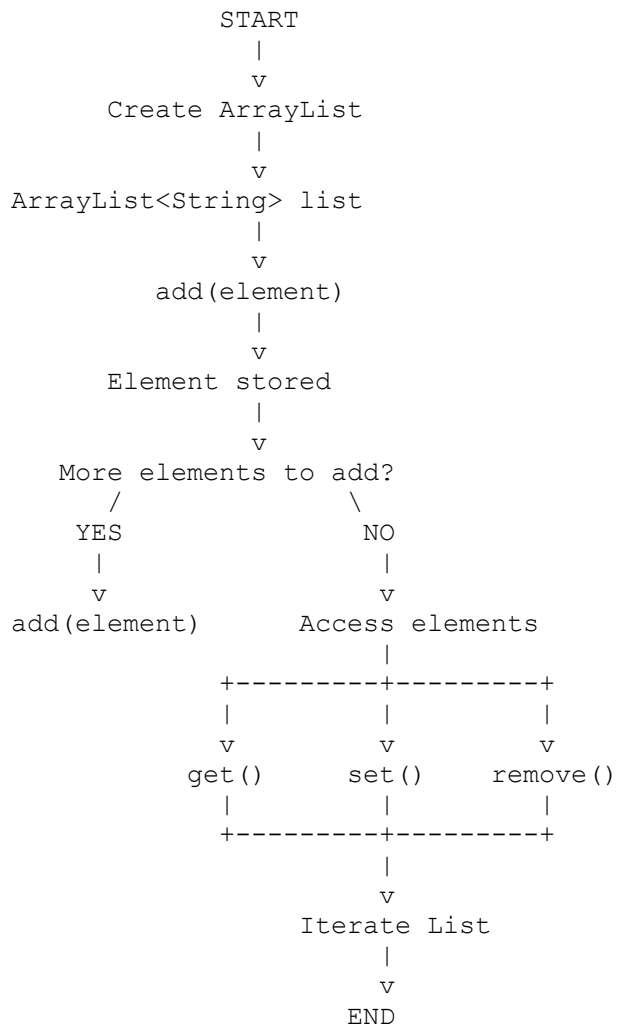
```
java.lang.Object
    |
    v
java.util.AbstractCollection
    |
    v
java.util.AbstractList
    |
    v
java.util.ArrayList
    |
    v
implements List
    |
    v
implements Collection
    |
    v
implements Iterable
```

More conceptually:

```
Iterable
    |
    v
Collection
```



3. ArrayList Flowchart



4. Basic Java Code Example

ArrayList.java

```
import java.util.ArrayList;
```

```

import java.util.List;

public class ArrayList {

    public static void main(String[] args) {

        // Create a List using ArrayList
        List<String> languages = new java.util.ArrayList<>();

        // Add elements
        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("JavaScript");

        // Display the list
        System.out.println("Languages: " + languages);

        // Access element using index
        System.out.println("First Language: " + languages.get(0));

        // Update an element
        languages.set(1, "Python 3");

        System.out.println("After Update: " + languages);

        // Remove an element
        languages.remove("C++");

        System.out.println("After Remove: " + languages);

        // Check whether an element exists
        System.out.println(
            "Contains Java: " + languages.contains("Java")
        );

        // Number of elements
        System.out.println(
            "Size: " + languages.size()
        );
    }
}

```

Here the class name is ArrayList, so the actual Java class is referenced as java.util.ArrayList to avoid a naming conflict.

Output

```
Languages: [Java, Python, C++, JavaScript]
```

```
First Language: Java
```

After Update: [Java, Python 3, C++, JavaScript]

After Remove: [Java, Python 3, JavaScript]

Contains Java: true

Size: 3

5. Better Example Using ArrayList Directly

For normal Java programming, it is preferable to give your own class a different name, such as ArrayListExample.

```
import java.util.ArrayList;

public class ArrayListExample {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        names.add("Anurag");
        names.add("Rahul");
        names.add("Amit");
        names.add("Anurag"); // Duplicate allowed

        System.out.println("Names: " + names);

        System.out.println("Element at index 0: " + names.get(0));

        names.set(1, "Rohit");

        System.out.println("After set(): " + names);

        names.remove(2);

        System.out.println("After remove(): " + names);

        System.out.println("Size: " + names.size());

    }
}
```

Output

Names: [Anurag, Rahul, Amit, Anurag]

Element at index 0: Anurag

After set(): [Anurag, Rohit, Amit, Anurag]

After remove(): [Anurag, Rohit, Anurag]

Size: 3

Notice that **Anurag appears twice**. This demonstrates that ArrayList allows duplicate elements.

6. Important ArrayList Methods

Method	Purpose	Example
add()	Adds an element	list.add("Java")
add(index, element)	Adds at specific position	list.add(1, "Python")
get()	Retrieves element	list.get(0)
set()	Replaces element	list.set(0, "C++")
remove()	Removes element	list.remove(0)
contains()	Checks element	list.contains("Java")
size()	Returns number of elements	list.size()
isEmpty()	Checks whether list is empty	list.isEmpty()
clear()	Removes all elements	list.clear()
indexOf()	Finds first occurrence	list.indexOf("Java")
lastIndexOf()	Finds last occurrence	list.lastIndexOf("Java")

7. ArrayList Index Concept

ArrayList uses **zero-based indexing**.

ArrayList:

```
Index:      0          1          2          3
           |          |          |          |
           v          v          v          v
+-----+ +-----+ +-----+ +-----+
| Java  | | Python| | C++   | | SQL   |
```

```

+-----+ +-----+ +-----+ +-----+
      ^
      |
get(0)
returns "Java"

```

For example:

```
ArrayList<String> list = new ArrayList<>();
```

```
list.add("Java");
list.add("Python");
list.add("C++");
```

```
System.out.println(list.get(0));
System.out.println(list.get(1));
System.out.println(list.get(2));
```

Output:

```
Java
Python
C++
```

8. ArrayList Characteristics

Characteristic	ArrayList
Introduced	Java 1.2
Year	1998
Package	java.util
Type	Class
Implements	List
Maintains insertion order	Yes
Allows duplicates	Yes
Allows index-based access	Yes
Allows null	Yes

Dynamic/resizable	Yes
Fast random access	Yes
Thread-safe by default	No
Best suited for	Frequent reading/access by index

Key Concept

```

ArrayList
|
+--> Ordered
|
+--> Duplicate values allowed
|
+--> Index-based access
|
+--> Dynamically resizable
|
+--> Fast get(index)
|
+--> Not synchronized by default

```

In one sentence: ArrayList is a resizable-array implementation of the List interface that maintains insertion order, permits duplicates, and provides efficient index-based access.

Java Set / HashSet

Java Version: 1.2

Year: 1998

Feature: Set / HashSet

UniqueClassName: HashSet

1. Definition

Set

Set is an interface in the Java Collections Framework that represents a collection that **does not allow duplicate elements**.

Important characteristics of Set:

- Does **not permit duplicate elements**.
- Generally represents a collection of unique elements.
- Does not provide index-based access like List.
- Common implementations include HashSet, LinkedHashSet, and TreeSet.

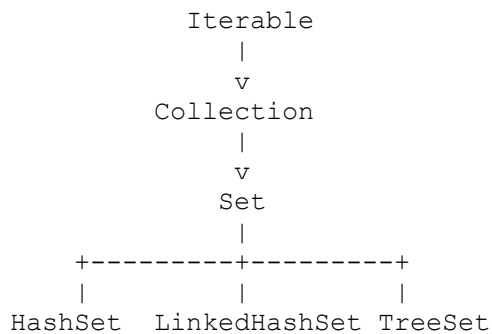
HashSet

HashSet is a class that implements the Set interface and stores **unique elements using hashing**.

It was introduced as part of the **Java Collections Framework in Java 1.2 (1998)**.

```
Set
|
+---- HashSet
|
+---- LinkedHashSet
|
+---- TreeSet
```

2. HashSet Hierarchy



More specifically:

```

java.lang.Object
    |
    v
AbstractCollection
    |
    v
    Set
    |
    v
    HashSet

```

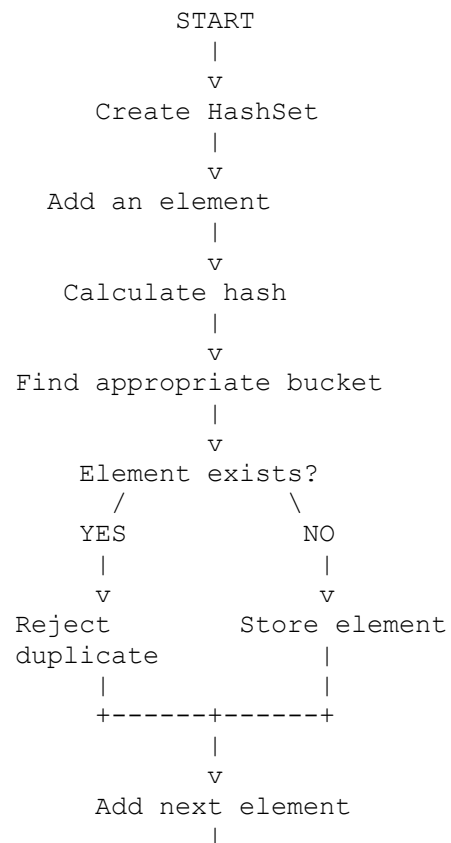
HashSet directly extends AbstractSet:

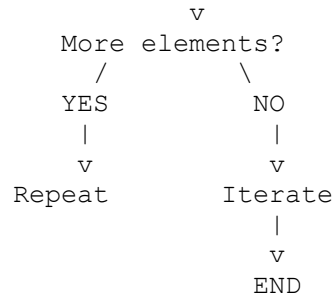
```
Object
|
v
AbstractCollection
|
v
AbstractSet
|
v
HashSet
```

And:

```
HashSet
|
+--> implements Set
      |
      +--> Collection
            |
            +--> Iterable
```

3. HashSet Flowchart





4. Basic Java Code Example

HashSetExample.java

```
import java.util.HashSet;

public class HashSetExample {

    public static void main(String[] args) {

        // Create HashSet
        HashSet<String> languages = new HashSet<>();

        // Add elements
        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("JavaScript");

        // Duplicate element
        languages.add("Java");

        // Display HashSet
        System.out.println("Languages: " + languages);

        // Check whether an element exists
        System.out.println(
            "Contains Java: " + languages.contains("Java")
        );

        // Remove an element
        languages.remove("C++");

        System.out.println(
            "After removing C++: " + languages
        );

        // Size
        System.out.println(
            "Size: " + languages.size()
        );
    }
}
```

```
}
```

Output

The exact order can vary because HashSet does **not guarantee insertion order**.

```
Languages: [Java, C++, Python, JavaScript]
```

```
Contains Java: true
```

```
After removing C++: [Java, Python, JavaScript]
```

```
Size: 3
```

The important point is that adding "Java" twice still results in **only one "Java"**.

5. Demonstrating Duplicate Removal

```
import java.util.HashSet;

public class HashSetDuplicateExample {

    public static void main(String[] args) {

        HashSet<Integer> numbers = new HashSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(20);
        numbers.add(10);

        System.out.println("Numbers: " + numbers);
        System.out.println("Size: " + numbers.size());
    }
}
```

Output:

```
Numbers: [10, 20, 30]
```

```
Size: 3
```

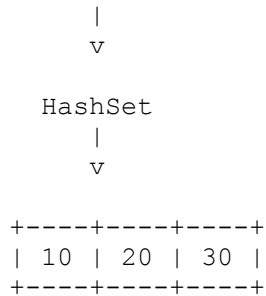
What happened?

Input:

```

10
20
30
20 <-- duplicate
10 <-- duplicate

```



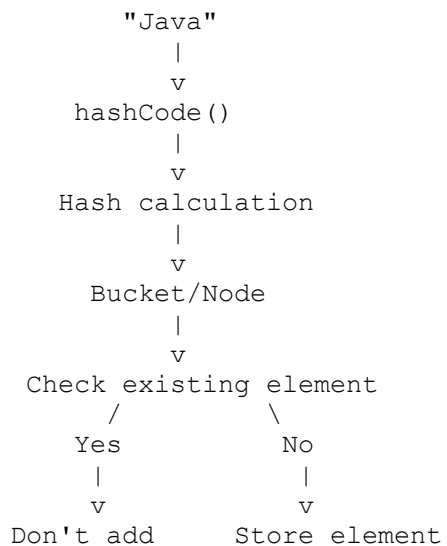
Only unique values remain.

6. HashSet Internal Concept

Conceptually, when you execute:

```
set.add("Java");
```

Java uses the object's **hash code** to determine where the element should be stored.



For modern Java implementations, hash-based collections use hash tables with bucket structures. The exact internal implementation details have evolved across Java releases.

7. Important HashSet Methods

Method	Purpose	Example
add()	Adds an element	set.add("Java")
remove()	Removes an element	set.remove("Java")
contains()	Checks whether element exists	set.contains("Java")
size()	Returns number of elements	set.size()
isEmpty()	Checks whether set is empty	set.isEmpty()
clear()	Removes all elements	set.clear()
iterator()	Creates an iterator	set.iterator()
addAll()	Adds elements from another collection	set.addAll(list)
removeAll()	Removes matching elements	set.removeAll(otherSet)
retainAll()	Retains common elements	set.retainAll(otherSet)

8. Iterating Through HashSet

```
import java.util.HashSet;

public class HashSetIteration {

    public static void main(String[] args) {

        HashSet<String> languages = new HashSet<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("JavaScript");
```

```

        for (String language : languages) {
            System.out.println(language);
        }
    }
}

```

Possible output:

```

Java
C++
Python
JavaScript

```

Important: Do not depend on this order. HashSet does not guarantee insertion order.

9. Set vs List

Feature	List	Set
Duplicate elements	Yes	No
Insertion order	Generally maintained	Depends on implementation
Index-based access	Yes	No
Example	ArrayList	HashSet
Best use	Ordered data	Unique data

Example:

List:

```

[Java, Python, Java, C++]
      ^
      |
Duplicate allowed

```

HashSet:

```

[Java, Python, C++]
      ^
      |
Duplicate removed

```

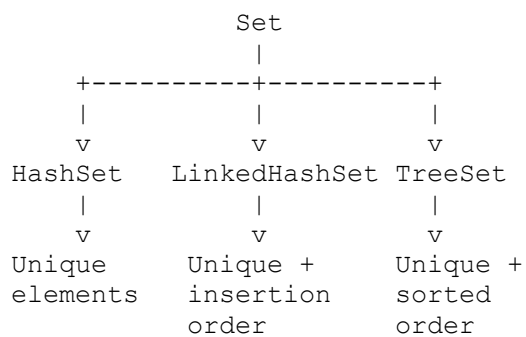
10. HashSet vs LinkedHashSet vs TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates	No	No	No
Insertion order	No guarantee	Yes	No
Sorted order	No	No	Yes
Typical purpose	Fast unique-element storage	Unique + insertion order	Unique + sorted order
Index access	No	No	No
Introduced	Java 1.2	Java 1.4	Java 1.2

Example:

```
Set<Integer> hashSet      = new HashSet<>();
Set<Integer> linkedHashSet = new java.util.LinkedHashSet<>();
Set<Integer> treeSet      = new java.util.TreeSet<>();
```

Conceptually:



11. Key Characteristics of HashSet

Characteristic	HashSet
Java version	1.2
Year	1998
Package	java.util

Type	Class
Implements	Set
Duplicate values	Not allowed
Insertion order	Not guaranteed
Index-based access	No
null	Permits one null element
Hashing	Yes
Thread-safe by default	No
Primary purpose	Store unique elements

Key Concept

```

HashSet
|
+--> Unique elements
|
+--> No duplicates
|
+--> Hash-based storage
|
+--> No index-based access
|
+--> No guaranteed iteration order
|
+--> One null element permitted
|
+--> Fast average-case basic operations

```

In one sentence: HashSet is a Java Collections Framework class that uses hashing to store **unique elements**, does not guarantee iteration order, and provides efficient average-case operations such as `add()`, `remove()`, and `contains()`.

Java Map / HashMap

Java Version: 1.2

Year: 1998

Feature: Map / HashMap

UniqueClassName: HashMap

1. Definition

Map

Map is an interface in the Java Collections Framework that represents a collection of **key-value pairs**.

Each entry consists of:

Key → Value

For example:

101 → "Anurag"

102 → "Rahul"

103 → "Amit"

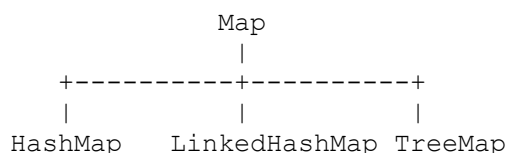
Important characteristics of Map:

- Stores data as **key-value pairs**.
- Each key is **unique**.
- Multiple keys can have the same value.
- A key maps to at most one value.
- Provides operations such as put(), get(), remove(), and containsKey().
- Map does **not extend Collection**.

HashMap

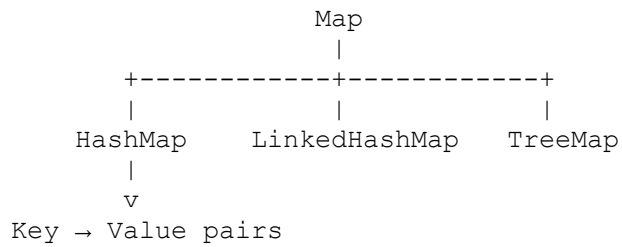
HashMap is a hash-table-based implementation of the Map interface.

It was introduced with the **Java Collections Framework in Java 1.2 (1998)**.

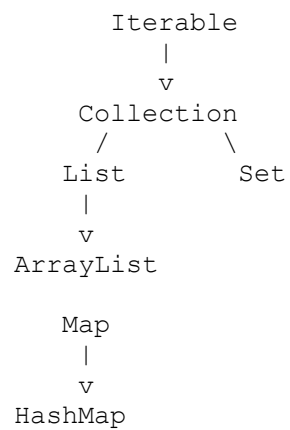


2. Map Hierarchy

The important conceptual hierarchy is:

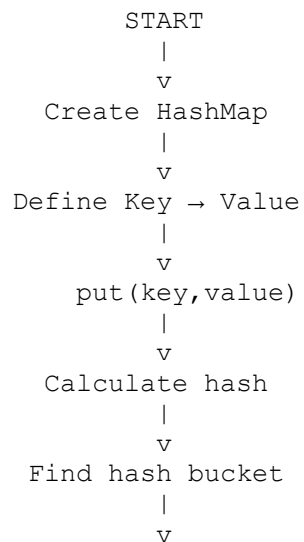


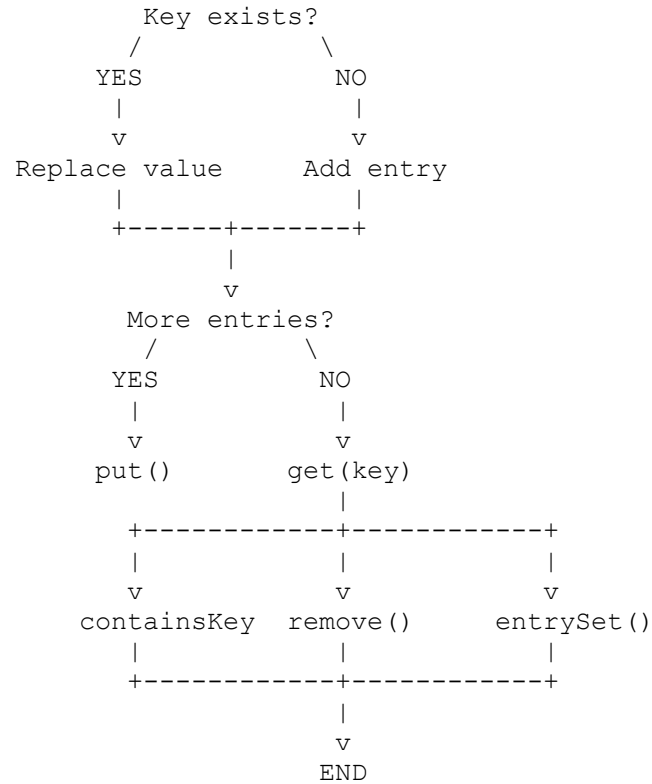
Unlike List and Set:



Important: Map is part of the Collections Framework, but it is **not a subtype of Collection**.

3. HashMap Flowchart





4. Basic Java Code Example

HashMapExample.java

```

import java.util.HashMap;
import java.util.Map;

public class HashMapExample {

    public static void main(String[] args) {

        // Create HashMap
        Map<Integer, String> students = new HashMap<>();

        // Add key-value pairs
        students.put(101, "Anurag");
        students.put(102, "Rahul");
        students.put(103, "Amit");
        students.put(104, "Rohit");

        // Display map
        System.out.println("Students: " + students);

        // Get value using key
        System.out.println(
            "Student with ID 101: " + students.get(101)
        );
    }
}

```

```

    );

    // Check whether key exists
    System.out.println(
        "Contains ID 102: " + students.containsKey(102)
    );

    // Update value
    students.put(102, "Vijay");

    System.out.println(
        "After Update: " + students
    );

    // Remove entry
    students.remove(103);

    System.out.println(
        "After Remove: " + students
    );

    // Size
    System.out.println(
        "Size: " + students.size()
    );
}
}

```

Possible Output

```

Students: {101=Anurag, 102=Rahul, 103=Amit, 104=Rohit}

Student with ID 101: Anurag

Contains ID 102: true

After Update: {101=Anurag, 102=Vijay, 103=Amit, 104=Rohit}

After Remove: {101=Anurag, 102=Vijay, 104=Rohit}

Size: 3

```

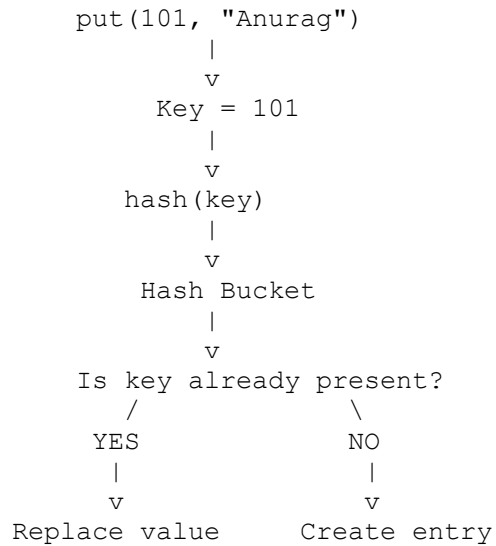
Note: The iteration/display order of a HashMap is **not guaranteed**, so the order shown can differ.

5. Understanding put()

Consider:

```
students.put(101, "Anurag");
```

Conceptually:



The entry becomes:

```

+-----+-----+
| Key | Value |
+-----+-----+
| 101 | Anurag |
+-----+-----+

```

6. Duplicate Key Example

A HashMap **does not allow duplicate keys**.

```

import java.util.HashMap;

public class HashMapDuplicateKey {

    public static void main(String[] args) {

        HashMap<Integer, String> map = new HashMap<>();

        map.put(101, "Anurag");
        map.put(102, "Rahul");

        // Same key again
        map.put(101, "Amit");

        System.out.println(map);
    }
}

```

Output:

{101=Amit, 102=Rahul}

Why?

First:

101 → Anurag

Second:

101 → Amit

↓

Same key

↓

Existing value replaced

↓

101 → Amit

So:

Duplicate key → old value is replaced.

7. Duplicate Values Are Allowed

Although keys must be unique, values can be duplicated.

```
HashMap<Integer, String> map = new HashMap<>();
```

```
map.put(101, "Java");  
map.put(102, "Python");  
map.put(103, "Java");
```

Conceptually:

101 → Java
102 → Python
103 → Java

This is valid because the **keys are different**.

8. Important HashMap Methods

Method	Purpose	Example
put()	Adds/updates key-value pair	map.put(101, "Java")
get()	Gets value using key	map.get(101)
remove()	Removes entry	map.remove(101)
containsKey()	Checks key	map.containsKey(101)
containsValue()	Checks value	map.containsValue("Java")
size()	Number of entries	map.size()
isEmpty()	Checks whether map is empty	map.isEmpty()
clear()	Removes all entries	map.clear()
keySet()	Returns keys	map.keySet()
values()	Returns values	map.values()
entrySet()	Returns key-value entries	map.entrySet()
putAll()	Copies entries from another map	map.putAll(otherMap)

9. Iterating Through a HashMap

The preferred way to process key-value pairs is often `entrySet()`:

```
import java.util.HashMap;
import java.util.Map;

public class HashMapIteration {

    public static void main(String[] args) {

        Map<Integer, String> students = new HashMap<>();

        students.put(101, "Anurag");
```

```

students.put(102, "Rahul");
students.put(103, "Amit");

for (Map.Entry<Integer, String> entry : students.entrySet()) {

    System.out.println(
        "ID = " + entry.getKey()
        + ", Name = " + entry.getValue()
    );
}
}
}

```

Possible output:

```

ID = 101, Name = Anurag
ID = 102, Name = Rahul
ID = 103, Name = Amit

```

Again, the actual order is not guaranteed by HashMap.

10. HashMap with null

HashMap permits:

- **One null key**
- **Multiple null values**

Example:

```

import java.util.HashMap;

public class HashMapNullExample {

    public static void main(String[] args) {

        HashMap<Integer, String> map = new HashMap<>();

        map.put(null, "Unknown");
        map.put(101, null);
        map.put(102, null);

        System.out.println(map);
    }
}

```

Conceptually:

null → Unknown

101 → null

102 → null

11. Map vs List vs Set

Feature	List	Set	Map
Stores	Elements	Unique elements	Key-value pairs
Duplicate elements	Yes	No	Values: Yes
Duplicate keys	—	—	No
Index access	Yes	No	No
Example	ArrayList	HashSet	HashMap
Main access	Index	Element	Key

Conceptually:

List

|

---> 0 → Java

---> 1 → Python

---> 2 → C++

Set

|

---> Java

---> Python

---> C++

Map

|

---> 101 → Java

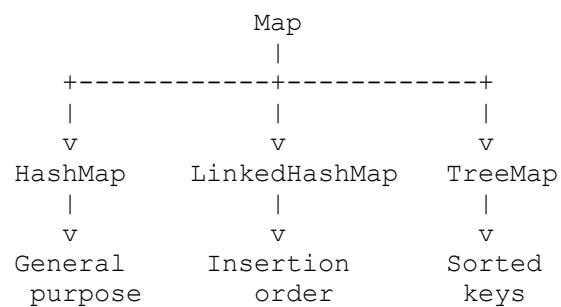
---> 102 → Python

---> 103 → C++

12. HashMap vs LinkedHashMap vs TreeMap

Feature	HashMap	LinkedHashMap	TreeMap
Duplicate keys	No	No	No
Insertion order	No guarantee	Yes	No
Sorted keys	No	No	Yes
Hashing	Yes	Yes	No; tree-based
null key	One allowed	One allowed	Generally not allowed with natural ordering
Main purpose	Fast general-purpose map	Map + insertion order	Sorted map

Conceptually:



13. Key Characteristics of HashMap

Characteristic	HashMap
Java version	1.2
Year	1998
Package	java.util
Type	Class
Implements	Map
Data structure concept	Hash table

Stores	Key-value pairs
Duplicate keys	Not allowed
Duplicate values	Allowed
Insertion order	Not guaranteed
null key	One allowed
null values	Multiple allowed
Index-based access	No
Thread-safe by default	No
Average basic operation performance	Typically $O(1)$
Main purpose	Fast key-based lookup

Key Concept

```

HashMap
|
+--> Key → Value
|
+--> Unique keys
|
+--> Duplicate values allowed
|
+--> Hash-based lookup
|
+--> No guaranteed order
|
+--> One null key
|
+--> Multiple null values
|
+--> Fast average-case lookup

```

In one sentence: HashMap is a Java 1.2 Collections Framework class that stores **key-value pairs using hashing**, does not allow duplicate keys, permits duplicate values, and provides efficient average-case key-based operations such as `put()`, `get()`, `remove()`, and `containsKey()`.

Java TreeSet

Java Version: 1.2

Year: 1998

Feature: TreeSet

UniqueClassName: TreeSet

1. Definition

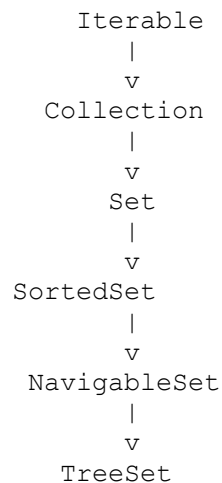
TreeSet is a class in the Java Collections Framework that implements the **Set interface** and stores **unique elements in sorted order**.

It is based on a **tree data structure** and, in current Java implementations, is backed by a TreeMap.

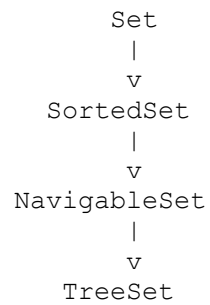
Main characteristics

- Does **not allow duplicate elements**.
 - Maintains elements in **ascending/natural sorted order** by default.
 - Can use a custom Comparator for different sorting.
 - Does **not provide index-based access**.
 - Provides efficient basic operations such as add(), remove(), and contains(), typically **$O(\log n)$** .
 - Generally does not permit null when using natural ordering.
 - Introduced in **Java 1.2 (1998)**.
-

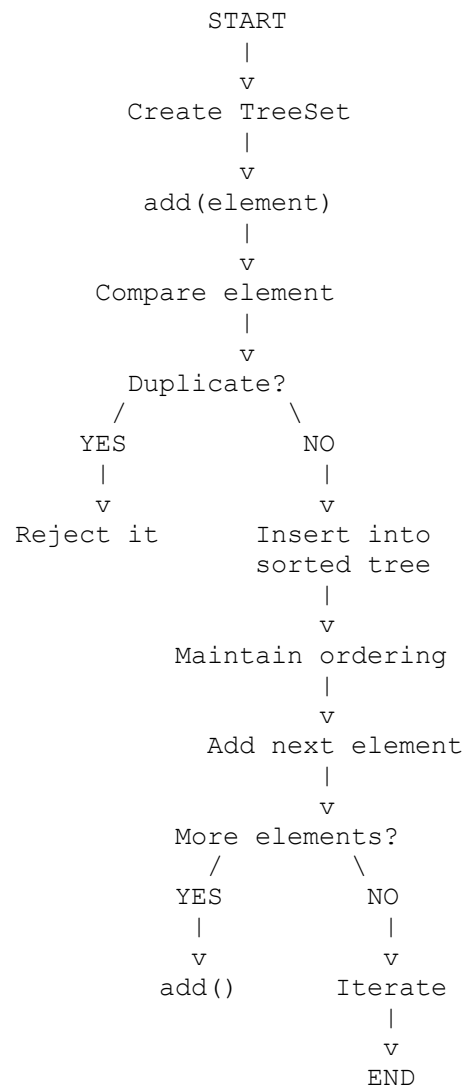
2. TreeSet Hierarchy



A simplified view:



3. TreeSet Flowchart



4. Basic Java Code Example

TreeSetExample.java

```
import java.util.TreeSet;

public class TreeSetExample {

    public static void main(String[] args) {

        // Create TreeSet
        TreeSet<Integer> numbers = new TreeSet<>();

        // Add elements
        numbers.add(50);
        numbers.add(20);
        numbers.add(40);
        numbers.add(10);
        numbers.add(30);

        // Duplicate
        numbers.add(20);

        // Display TreeSet
        System.out.println("Numbers: " + numbers);

        // Check element
        System.out.println(
            "Contains 30: " + numbers.contains(30)
        );

        // Remove element
        numbers.remove(40);

        System.out.println(
            "After removing 40: " + numbers
        );

        // First and last elements
        System.out.println(
            "First: " + numbers.first()
        );

        System.out.println(
            "Last: " + numbers.last()
        );
    }
}
```

Output

Numbers: [10, 20, 30, 40, 50]

Contains 30: true

After removing 40: [10, 20, 30, 50]

First: 10

Last: 50

Notice two important things:

Input:

50, 20, 40, 10, 30, 20

|
v

TreeSet

|
+--> Removes duplicate 20
|
+--> Sorts elements
|
v

10, 20, 30, 40, 50

5. TreeSet with Strings

TreeSet also sorts strings according to their natural ordering.

```
import java.util.TreeSet;

public class TreeSetStringExample {

    public static void main(String[] args) {

        TreeSet<String> languages = new TreeSet<>();

        languages.add("Python");
        languages.add("Java");
        languages.add("C++");
        languages.add("JavaScript");
        languages.add("Java");

        System.out.println("Languages: " + languages);
    }
}
```

Output:

Languages: [C++, Java, JavaScript, Python]

The second "Java" is ignored because TreeSet does not permit duplicates.

6. TreeSet Sorting Concept

By default, TreeSet uses **natural ordering**.

For numbers:

Input:

50
10
40
20
30

|
v

TreeSet

|
v

Natural Ordering

|
v

10
20
30
40
50

For strings:

Input:

"Python"
"Java"
"C++"
"JavaScript"

|
v

TreeSet

|
v

"C++"
"Java"
"JavaScript"
"Python"

7. TreeSet Important Methods

Method	Purpose	Example
add()	Adds an element	set.add(10)
remove()	Removes an element	set.remove(10)
contains()	Checks an element	set.contains(10)
size()	Returns number of elements	set.size()
isEmpty()	Checks whether empty	set.isEmpty()
clear()	Removes all elements	set.clear()
first()	Returns smallest element	set.first()
last()	Returns largest element	set.last()
higher()	Element strictly greater than given element	set.higher(20)
lower()	Element strictly smaller than given element	set.lower(20)
ceiling()	Smallest element $\geq$ given element	set.ceiling(20)
floor()	Largest element $\leq$ given element	set.floor(20)
pollFirst()	Removes first element	set.pollFirst()
pollLast()	Removes last element	set.pollLast()

8. first(), last(), higher(), lower()

```
import java.util.TreeSet;

public class TreeSetNavigation {

    public static void main(String[] args) {

        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);
        numbers.add(50);

        System.out.println("Numbers: " + numbers);

        System.out.println("First: " + numbers.first());
        System.out.println("Last: " + numbers.last());

        System.out.println("Higher than 20: "
            + numbers.higher(20));

        System.out.println("Lower than 20: "
            + numbers.lower(20));

        System.out.println("Ceiling of 25: "
            + numbers.ceiling(25));

        System.out.println("Floor of 25: "
            + numbers.floor(25));
    }
}
```

Output:

Numbers: [10, 20, 30, 40, 50]

First: 10
Last: 50
Higher than 20: 30
Lower than 20: 10
Ceiling of 25: 30
Floor of 25: 20

9. TreeSet with Custom Sorting

You can provide a Comparator to change the sorting order.

Descending Order

```

import java.util.Comparator;
import java.util.TreeSet;

public class TreeSetDescending {

    public static void main(String[] args) {

        TreeSet<Integer> numbers =
            new TreeSet<>(Comparator.reverseOrder());

        numbers.add(10);
        numbers.add(50);
        numbers.add(20);
        numbers.add(40);
        numbers.add(30);

        System.out.println(numbers);
    }
}

```

Output:

[50, 40, 30, 20, 10]

Conceptually:

Natural Order:

10 → 20 → 30 → 40 → 50

Comparator.reverseOrder() :

50 → 40 → 30 → 20 → 10

10. TreeSet vs HashSet

Feature	HashSet	TreeSet
Duplicate elements	No	No
Ordering	No guaranteed order	Sorted
Natural ordering	No	Yes
Custom comparator	No ordering comparator	Yes
Data structure concept	Hash table	Balanced tree

Basic operation	Typically $O(1)$ average	Typically $O(\log n)$
Index access	No	No
null	One null generally allowed	Generally not with natural ordering
Main purpose	Fast unique-element storage	Unique + sorted elements

HashSet

50, 10, 40, 20, 30

|
v

No guaranteed order

TreeSet

50, 10, 40, 20, 30

|
v

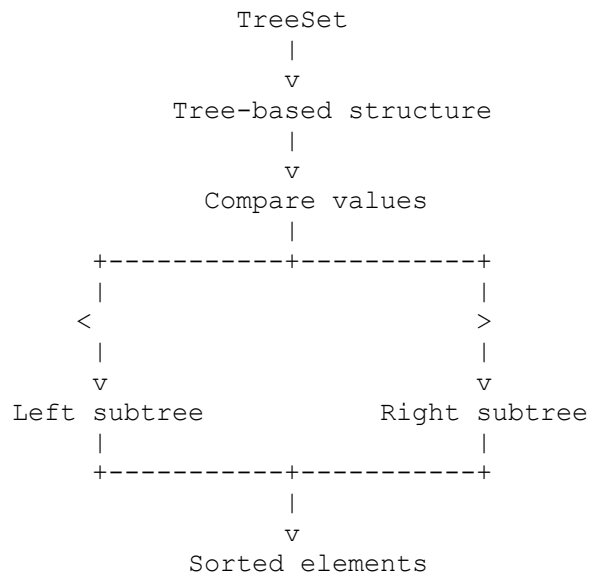
10, 20, 30, 40, 50

11. TreeSet vs ArrayList

Feature	ArrayList	TreeSet
Interface	List	NavigableSet / SortedSet
Duplicates	Allowed	Not allowed
Insertion order	Maintained	Not the defining order
Sorted automatically	No	Yes
Index access	Yes	No
Random access	Fast	Not index-based
Typical basic search	$O(n)$	$O(\log n)$
Main purpose	Ordered collection with duplicates	Unique sorted collection

12. TreeSet Internal Concept

Conceptually:



Modern Java's TreeSet is implemented using a TreeMap, which uses a **Red-Black tree**.

13. Key Characteristics

Characteristic	TreeSet
Java Version	1.2
Year	1998
Package	java.util
Type	Class
Implements	NavigableSet, SortedSet, Set
Duplicate elements	Not allowed
Ordering	Sorted
Default ordering	Natural ordering

Custom ordering	Comparator supported
Index access	No
Basic operations	Typically $O(\log n)$
Thread-safe by default	No
null	Generally not supported with natural ordering
Main purpose	Unique sorted elements

Key Concept

```

TreeSet
|
+--> Unique elements
|
+--> No duplicates
|
+--> Automatically sorted
|
+--> Natural ordering
|
+--> Comparator supported
|
+--> No index-based access
|
+--> Tree-based implementation
|
+-->  $O(\log n)$  typical basic operations

```

In one sentence: TreeSet is a Java 1.2 Collections Framework class that stores **unique elements in sorted order** and provides tree-based operations such as `add()`, `remove()`, `contains()`, `first()`, `last()`, `higher()`, and `lower()`.

Java TreeMap

Java Version: 1.2

Year: 1998

Feature: TreeMap

UniqueClassName: TreeMap

1. Definition

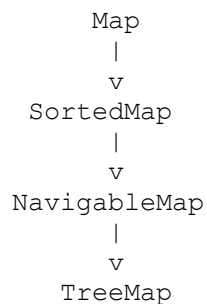
TreeMap is a class in the Java Collections Framework that implements the **NavigableMap** interface and stores **key-value pairs in sorted order of keys**.

It is a tree-based implementation of Map. In current Java implementations, TreeMap is based on a **Red-Black tree**.

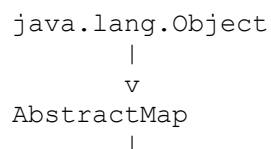
Main characteristics

- Stores data as **key-value pairs**.
 - Does **not allow duplicate keys**.
 - Allows duplicate values.
 - Maintains keys in **sorted order**.
 - Uses **natural ordering** by default.
 - Supports custom ordering using a Comparator.
 - Provides navigation methods such as `firstKey()`, `lastKey()`, `higherKey()`, `lowerKey()`, `ceilingKey()`, and `floorKey()`.
 - Basic operations such as `put()`, `get()`, `remove()`, and `containsKey()` typically take **$O(\log n)$** time.
 - Introduced in **Java 1.2 (1998)**.
-

2. TreeMap Hierarchy



More completely:

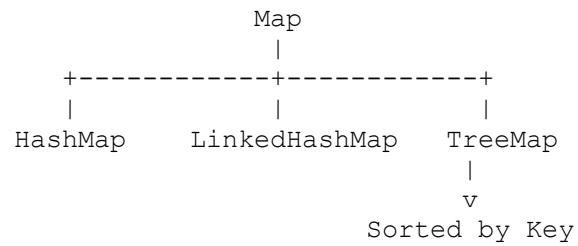


```

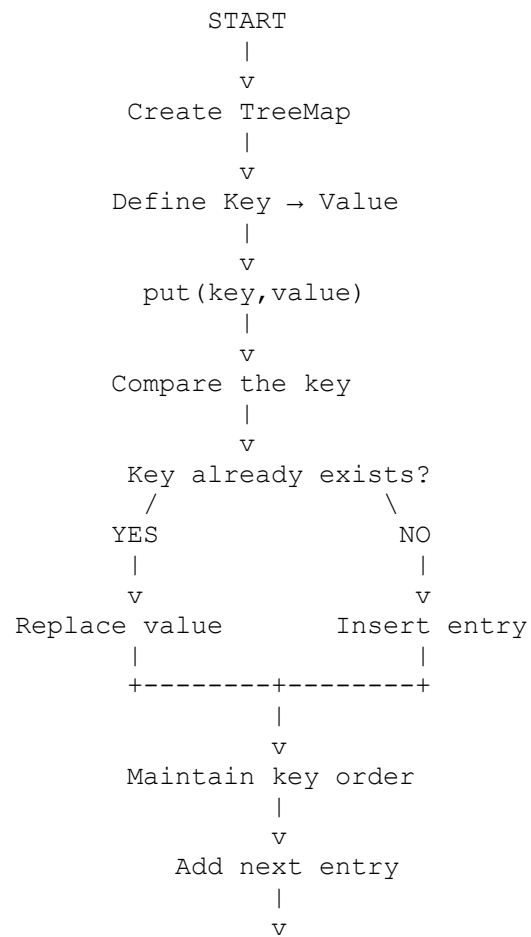
v
TreeMap
|
+---- implements Map
|
+---- implements SortedMap
|
+---- implements NavigableMap

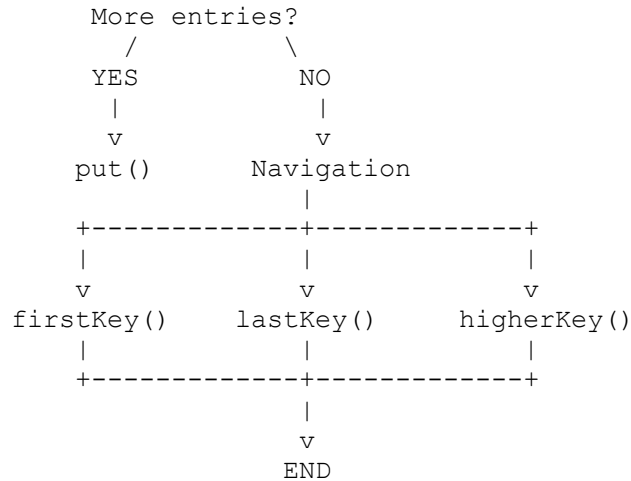
```

Conceptually:



3. TreeMap Flowchart





```
import java.util.TreeMap;

public class TreeMapExample {

    public static void main(String[] args) {

        // Create TreeMap
        TreeMap<Integer, String> students = new TreeMap<>();

        // Add key-value pairs
        students.put(103, "Amit");
        students.put(101, "Anurag");
        students.put(104, "Rohit");
        students.put(102, "Rahul");

        // Display TreeMap
        System.out.println("Students: " + students);

        // Get value using key
        System.out.println(
            "Student with ID 101: " + students.get(101)
        );

        // Check whether key exists
        System.out.println(
            "Contains ID 102: " + students.containsKey(102)
        );

        // Remove entry
        students.remove(103);

        System.out.println(
```

```

        "After removing ID 103: " + students
    );

    // First and last keys
    System.out.println(
        "First Key: " + students.firstKey()
    );

    System.out.println(
        "Last Key: " + students.lastKey()
    );
}
}

```

Output

```

Students: {101=Anurag, 102=Rahul, 103=Amit, 104=Rohit}

Student with ID 101: Anurag

Contains ID 102: true

After removing ID 103: {101=Anurag, 102=Rahul, 104=Rohit}

First Key: 101

Last Key: 104

```

The important point is that the entries are displayed according to **sorted key order**, not insertion order.

5. TreeMap Sorting Concept

Suppose we insert:

```

map.put(50, "A");
map.put(10, "B");
map.put(40, "C");
map.put(20, "D");
map.put(30, "E");

```

Insertion order:

```

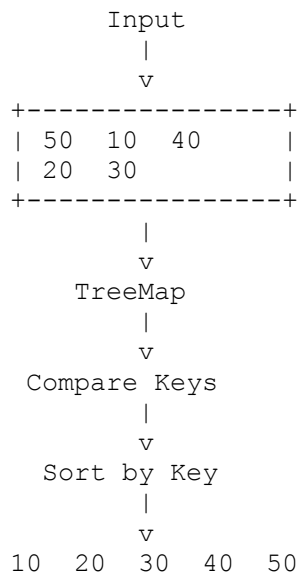
50 → A
10 → B
40 → C
20 → D
30 → E

```

TreeMap sorts the keys:

10 → B
20 → D
30 → E
40 → C
50 → A

Flow:



6. Duplicate Key Example

Like every Map, a TreeMap does not allow duplicate keys.

```
import java.util.TreeMap;

public class TreeMapDuplicateKey {

    public static void main(String[] args) {

        TreeMap<Integer, String> map = new TreeMap<>();

        map.put(101, "Anurag");
        map.put(102, "Rahul");

        // Same key
        map.put(101, "Amit");

        System.out.println(map);
    }
}
```

```
}
```

Output:

```
{101=Amit, 102=Rahul}
```

Conceptually:

```
101 → Anurag
      |
      | put(101, "Amit")
      v
101 → Amit
```

The **old value is replaced**.

7. Duplicate Values Are Allowed

Different keys can have the same value:

```
TreeMap<Integer, String> map = new TreeMap<>();
```

```
map.put(101, "Java");
map.put(102, "Python");
map.put(103, "Java");
```

Result:

```
101 → Java
102 → Python
103 → Java
```

This is valid because the keys are unique.

8. TreeMap Navigation Methods

One of the major advantages of TreeMap is its navigation functionality.

```
import java.util.TreeMap;

public class TreeMapNavigation {

    public static void main(String[] args) {

        TreeMap<Integer, String> map = new TreeMap<>();
```

```

        map.put(10, "A");
        map.put(20, "B");
        map.put(30, "C");
        map.put(40, "D");
        map.put(50, "E");

        System.out.println("Map: " + map);

        System.out.println("First Key: "
            + map.firstKey());

        System.out.println("Last Key: "
            + map.lastKey());

        System.out.println("Higher than 20: "
            + map.higherKey(20));

        System.out.println("Lower than 20: "
            + map.lowerKey(20));

        System.out.println("Ceiling of 25: "
            + map.ceilingKey(25));

        System.out.println("Floor of 25: "
            + map.floorKey(25));
    }
}

```

Output:

Map: {10=A, 20=B, 30=C, 40=D, 50=E}

First Key: 10
 Last Key: 50
 Higher than 20: 30
 Lower than 20: 10
 Ceiling of 25: 30
 Floor of 25: 20

Meaning

```

10 ---- 20 ---- 30 ---- 40 ---- 50
      |
      +--> higherKey(20) = 30
      |
      +--> lowerKey(20)  = 10

ceilingKey(25) = 30
floorKey(25)   = 20

```

9. Custom Sorting with Comparator

By default, keys are sorted in ascending natural order.

You can specify a custom comparator.

Descending Key Order

```
import java.util.Comparator;
import java.util.TreeMap;

public class TreeMapDescending {

    public static void main(String[] args) {

        TreeMap<Integer, String> students =
            new TreeMap<>(Comparator.reverseOrder());

        students.put(101, "Anurag");
        students.put(103, "Amit");
        students.put(102, "Rahul");
        students.put(104, "Rohit");

        System.out.println(students);
    }
}
```

Output:

{104=Rohit, 103=Amit, 102=Rahul, 101=Anurag}

Conceptually:

Natural order:

101 → 102 → 103 → 104

Reverse order:

104 → 103 → 102 → 101

10. Iterating Through TreeMap

Using entrySet()

```
import java.util.Map;
import java.util.TreeMap;

public class TreeMapIteration {

    public static void main(String[] args) {

        TreeMap<Integer, String> students = new TreeMap<>();

        students.put(103, "Amit");
        students.put(101, "Anurag");
        students.put(104, "Rohit");
        students.put(102, "Rahul");

        for (Map.Entry<Integer, String> entry
             : students.entrySet()) {

            System.out.println(
                "ID = " + entry.getKey()
                + ", Name = " + entry.getValue()
            );
        }
    }
}
```

Output:

```
ID = 101, Name = Anurag
ID = 102, Name = Rahul
ID = 103, Name = Amit
ID = 104, Name = Rohit
```

Here the keys are automatically traversed in sorted order.

11. Important TreeMap Methods

Method	Purpose	Example
put()	Adds/updates key-value pair	map.put(101, "Java")
get()	Gets value for key	map.get(101)

remove()	Removes entry	map.remove(101)
containsKey()	Checks key	map.containsKey(101)
containsValue()	Checks value	map.containsValue("Java")
size()	Number of entries	map.size()
isEmpty()	Checks whether empty	map.isEmpty()
clear()	Removes all entries	map.clear()
firstKey()	Smallest key	map.firstKey()
lastKey()	Largest key	map.lastKey()
higherKey()	Key strictly greater	map.higherKey(20)
lowerKey()	Key strictly smaller	map.lowerKey(20)
ceilingKey()	Smallest key $\geq$ given key	map.ceilingKey(25)
floorKey()	Largest key $\leq$ given key	map.floorKey(25)
keySet()	Returns keys	map.keySet()
values()	Returns values	map.values()
entrySet()	Returns entries	map.entrySet()

12. TreeMap with String Keys

```
import java.util.TreeMap;
```

```
public class TreeMapStringExample {
```

```
    public static void main(String[] args) {
```

```
        TreeMap<String, Integer> employees = new TreeMap<>();
```

```
        employees.put("Rahul", 101);
        employees.put("Anurag", 102);
        employees.put("Amit", 103);
        employees.put("Rohit", 104);
```

```
        System.out.println(employees);
```

```
}  
}
```

Output:

```
{Amit=103, Anurag=102, Rahul=101, Rohit=104}
```

The keys are sorted alphabetically according to their natural ordering.

13. TreeMap vs HashMap

Feature	HashMap	TreeMap
Interface	Map	NavigableMap / SortedMap / Map
Key-value pairs	Yes	Yes
Duplicate keys	No	No
Duplicate values	Yes	Yes
Key ordering	No guarantee	Sorted
Natural key ordering	No	Yes
Custom Comparator	Not for ordering	Yes
Basic operation	O(1) average	O(log n)
Navigation methods	Limited	Rich
null key	One generally allowed	Generally not with natural ordering
Main purpose	Fast key lookup	Sorted/navigable key-value data

HashMap

```
101 → A  
50  → B  
200 → C  
10  → D
```

No guaranteed key order

TreeMap

101 → A
50 → B
200 → C
10 → D

↓

10 → D
50 → B
101 → A
200 → C

14. TreeMap vs TreeSet

Both are tree-based sorted collections, but they store different kinds of data.

Feature	TreeSet	TreeMap
Stores	Elements	Key-value pairs
Duplicate	Elements not allowed	Keys not allowed
Sorting based on	Elements	Keys
Interface	NavigableSet	NavigableMap
Example	[10, 20, 30]	{10=A, 20=B, 30=C}

Conceptually:

TreeSet

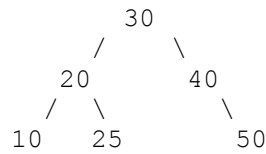
10
20
30
40

TreeMap

10 → A
20 → B
30 → C
40 → D

15. TreeMap Internal Structure

Conceptually, a TreeMap organizes entries according to their keys:



The exact tree shape depends on the implementation and balancing operations.

Modern Java TreeMap uses a **Red-Black tree**, which keeps the tree approximately balanced.

This provides:

put()	→ $O(\log n)$
get()	→ $O(\log n)$
remove()	→ $O(\log n)$
containsKey()	→ $O(\log n)$

16. Key Characteristics of TreeMap

Characteristic	TreeMap
Java Version	1.2
Year	1998
Package	java.util
Type	Class
Implements	Map, SortedMap, NavigableMap
Stores	Key-value pairs
Duplicate keys	Not allowed
Duplicate values	Allowed
Key ordering	Sorted
Default ordering	Natural ordering
Custom ordering	Comparator supported

Index access	No
Basic operations	Typically $O(\log n)$
Internal structure	Red-Black tree
Thread-safe by default	No
null key	Generally not supported with natural ordering
Main purpose	Sorted and navigable key-value data

Key Concept

```

TreeMap
|
+--> Key → Value
|
+--> Unique keys
|
+--> Duplicate values allowed
|
+--> Keys automatically sorted
|
+--> Natural ordering
|
+--> Comparator supported
|
+--> Navigation operations
|   |
|   +--> higherKey()
|   +--> lowerKey()
|   +--> ceilingKey()
|   +--> floorKey()
|
+--> Tree-based
|
+-->  $O(\log n)$  typical basic operations

```

In one sentence: TreeMap is a Java 1.2 Collections Framework class that stores **unique keys and their values in sorted key order**, using a tree-based structure to provide efficient and navigable key-value operations.

Java Iterator

Java Version: 1.2

Year: 1998

Feature: Iterator

UniqueClassName: Iterator

1. Definition

Iterator is an interface in the java.util package used to **traverse or iterate through elements of a collection one element at a time**.

The Iterator interface was introduced with the **Java Collections Framework in Java 1.2 (1998)**.

It provides a standard way to access collection elements without exposing the collection's internal data structure.

Main methods

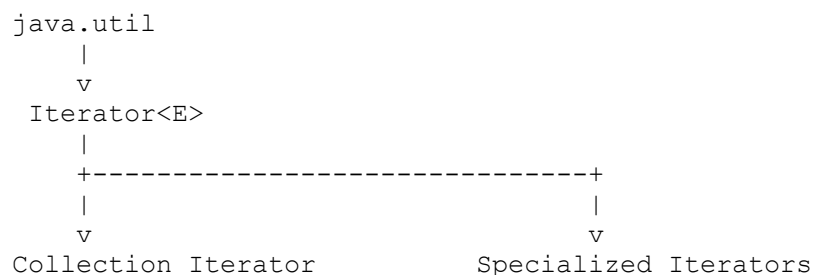
Method	Purpose
hasNext()	Checks whether another element exists
next()	Returns the next element
remove()	Removes the last element returned by the iterator, when supported

Basic pattern:

```
while (iterator.hasNext()) {  
    Object element = iterator.next();  
}
```

2. Iterator Hierarchy

Iterator is an interface rather than a collection class.



```

|
+--> ArrayList
+--> HashSet
+--> TreeSet
+--> LinkedList

```

```

|
+--> ListIterator

```

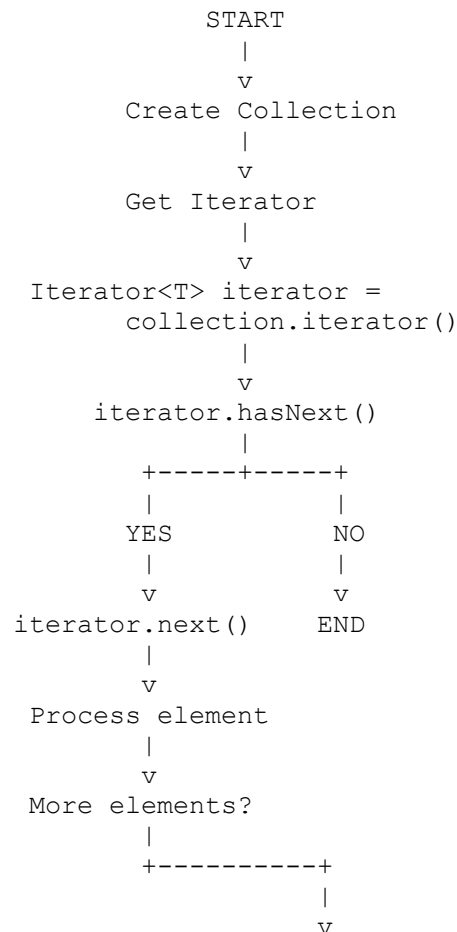
Conceptually:

```

Collection
|
v
iterator()
|
v
Iterator<E>
|
+--> hasNext()
+--> next()
+--> remove()

```

3. Iterator Flowchart



hasNext() again

4. Basic Java Code Example

IteratorExample.java

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorExample {

    public static void main(String[] args) {

        // Create a List
        List<String> languages = new ArrayList<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("JavaScript");

        // Create Iterator
        Iterator<String> iterator = languages.iterator();

        // Traverse the collection
        while (iterator.hasNext()) {

            String language = iterator.next();

            System.out.println(language);

        }
    }
}
```

Output

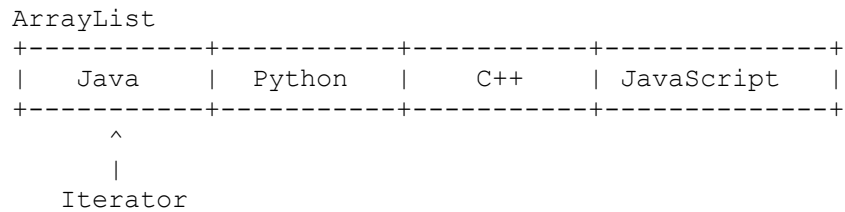
```
Java
Python
C++
JavaScript
```

5. How Iterator Works

Consider:

```
Iterator<String> iterator = languages.iterator();
```

Conceptually:



First:

```
iterator.hasNext();
```

returns:

true

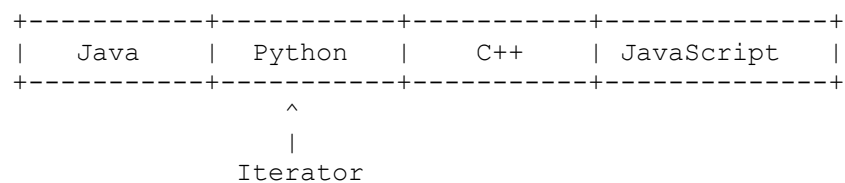
Then:

```
iterator.next();
```

returns:

Java

The iterator advances:



The process continues until:

```
iterator.hasNext();
```

returns:

false

6. hasNext() and next()

The two most important methods are:

hasNext()

Checks whether another element is available.

```
iterator.hasNext();
```

Returns:

true → another element exists

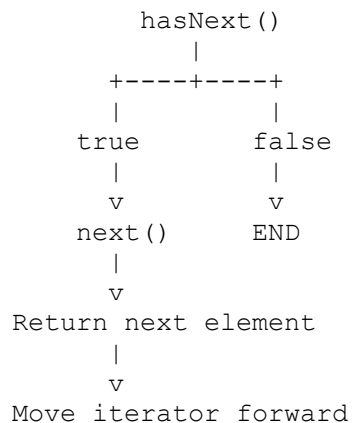
false → no more elements

next()

Returns the next element and advances the iterator.

```
String value = iterator.next();
```

Conceptually:



7. Iterator with HashSet

Iterator is not limited to List. It can also be used with a Set.

```
import java.util.HashSet;
import java.util.Iterator;
```

```

import java.util.Set;

public class IteratorHashSetExample {

    public static void main(String[] args) {

        Set<String> languages = new HashSet<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("JavaScript");

        Iterator<String> iterator =
            languages.iterator();

        while (iterator.hasNext()) {

            String language = iterator.next();

            System.out.println(language);

        }
    }
}

```

Possible output:

```

Java
C++
Python
JavaScript

```

The order can vary because HashSet does not guarantee iteration order.

8. Removing Elements Using Iterator

One important advantage of Iterator is that it provides a controlled way to remove the element currently being traversed.

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorRemoveExample {

    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<>();

        numbers.add(10);
        numbers.add(20);
    }
}

```

```

        numbers.add(30);
        numbers.add(40);
        numbers.add(50);

        Iterator<Integer> iterator =
            numbers.iterator();

        while (iterator.hasNext()) {

            Integer number = iterator.next();

            if (number == 30) {
                iterator.remove();
            }

        }

        System.out.println(numbers);
    }
}

```

Output:

[10, 20, 40, 50]

Flow

```

[10, 20, 30, 40, 50]
      |
      v
    next()
      |
      v
     30
      |
      v
  iterator.remove()
      |
      v
[10, 20, 40, 50]

```

9. Why Not Remove Directly?

Consider:

```

for (Integer number : numbers) {

    if (number == 30) {
        numbers.remove(number);
    }

}

```

Modifying many collections directly while using their iterator-based traversal can cause a `ConcurrentModificationException`.

The safer iterator-based pattern is:

```
Iterator<Integer> iterator = numbers.iterator();

while (iterator.hasNext()) {

    Integer number = iterator.next();

    if (number == 30) {
        iterator.remove();
    }
}
```

`Iterator.remove()` is specifically designed to remove the element most recently returned by that iterator, when the iterator supports removal.

10. Generic Iterator

Modern Java uses generics with `Iterator`.

```
Iterator<String> iterator;
```

This means:

```
Iterator<String>
  |
  v
Returns String
```

Example:

```
String language = iterator.next();
```

No explicit cast is normally required.

Without generics, older-style code could look like:

```
Iterator iterator;
Object value = iterator.next();
```

Generics provide compile-time type safety.

11. Iterator with TreeSet

```
import java.util.Iterator;
import java.util.TreeSet;

public class IteratorTreeSetExample {

    public static void main(String[] args) {

        TreeSet<Integer> numbers = new TreeSet<>();

        numbers.add(50);
        numbers.add(10);
        numbers.add(30);
        numbers.add(20);
        numbers.add(40);

        Iterator<Integer> iterator =
            numbers.iterator();

        while (iterator.hasNext()) {

            System.out.println(iterator.next());

        }

    }

}
```

Output:

```
10
20
30
40
50
```

Because TreeSet maintains sorted order, its iterator traverses the elements in sorted order.

12. Important Iterator Methods

Method	Return Type	Description
hasNext()	boolean	Checks whether another element exists
next()	E	Returns the next element
remove()	void	Removes the last element returned by next(), if supported

forEachRemaining()	void	Processes remaining elements; added in Java 8
---------------------------	------	--

Important version note

The original Iterator interface was introduced in **Java 1.2** with the Collections Framework.

forEachRemaining() was added later in **Java 8**.

13. Iterator vs Enhanced for Loop

Both can traverse collections.

Iterator

```
Iterator<String> iterator = languages.iterator();
while (iterator.hasNext()) {
    String language = iterator.next();
    System.out.println(language);
}
```

Enhanced for loop

```
for (String language : languages) {
    System.out.println(language);
}
```

Conceptually:

```
Enhanced for-loop
    |
    v
Uses Iterator-like traversal
    |
    v
hasNext()
    |
    v
next()
    |
    v
Process element
```

The enhanced for loop is generally simpler, while an explicit Iterator is useful when you need iterator-specific behavior such as `remove()`.

14. Iterator vs ListIterator

Feature	Iterator	ListIterator
Introduced	Java 1.2	Java 1.2
Works with	Collections generally	List only
Forward traversal	Yes	Yes
Backward traversal	No	Yes
<code>remove()</code>	Yes	Yes
<code>add()</code>	No	Yes
<code>set()</code>	No	Yes
<code>nextIndex()</code>	No	Yes
<code>previousIndex()</code>	No	Yes

Conceptually:

Iterator

START —————→ END

ListIterator

START —————→ END
←—————

15. Iterator Architecture

```
Collection
|
v
iterator()
|
v
Iterator<E>
|
```


Key Concept

```
Iterator
|
+--> Traverse collection
|
+--> hasNext()
|           |
|           +--> true / false
|
+--> next()
|           |
|           +--> Returns next element
|
+--> remove()
|
+--> Removes last returned element
```

In one sentence: Iterator is a Java 1.2 interface that provides a standard mechanism for **sequentially traversing collection elements**, primarily through `hasNext()`, `next()`, and optionally `remove()`.

Java Comparable

Java Version: 1.2

Year: 1998

Feature: Comparable

UniqueClassName: Comparable

Note: Comparable was introduced in **Java 1.2 (1998)** as part of the Collections Framework. It is used to define the **natural ordering** of objects.

1. Definition

`Comparable<T>` is a generic interface in the `java.lang` package that allows a class to define how its objects should be **naturally ordered**.

A class implements Comparable by overriding:

`compareTo(T other)`

The method returns:

Negative value → this object < other object
0 → this object == other object for ordering
Positive value → this object > other object

Basic structure:

```
class Student implements Comparable<Student> {  
  
    @Override  
    public int compareTo(Student other) {  
        return this.id - other.id;  
    }  
}
```

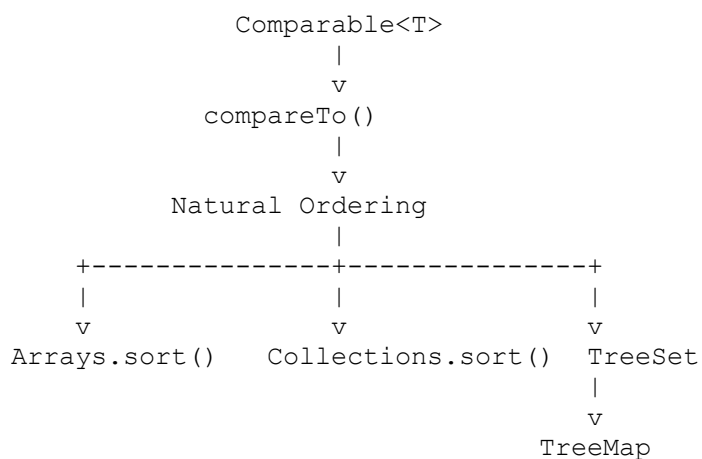
2. Comparable Hierarchy

Comparable is an interface in `java.lang`.

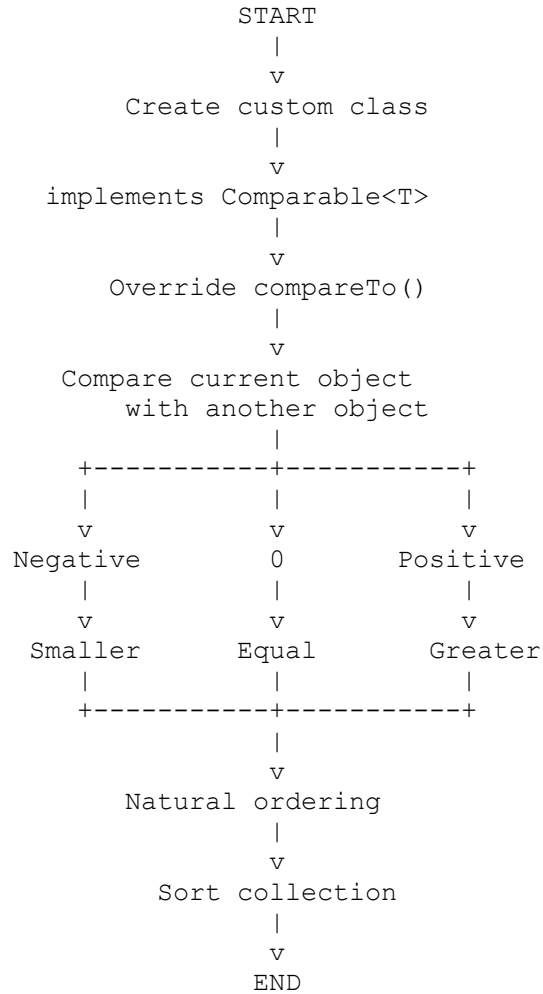
```
java.lang  
  |  
  v  
Comparable<T>  
  |  
  v  
compareTo(T object)
```

It is **not** part of the Collection hierarchy.

Instead, collection classes can use Comparable to determine the natural ordering of their elements.



3. Comparable Flowchart



4. Basic Java Code Example

ComparableExample.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

class Student implements Comparable<Student> {

    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

```

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.id, other.id);
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class ComparableExample {

    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        students.add(new Student(103, "Amit"));
        students.add(new Student(101, "Anurag"));
        students.add(new Student(102, "Rahul"));

        System.out.println("Before Sorting:");
        System.out.println(students);

        // Uses Student.compareTo()
        Collections.sort(students);

        System.out.println("\nAfter Sorting:");
        System.out.println(students);
    }
}

```

Output

```

Before Sorting:
[103 - Amit, 101 - Anurag, 102 - Rahul]

After Sorting:
[101 - Anurag, 102 - Rahul, 103 - Amit]

```

5. How compareTo() Works

The following code defines the natural ordering:

```
@Override  
public int compareTo(Student other) {  
    return Integer.compare(this.id, other.id);  
}
```

Suppose:

Student A $\rightarrow$ id = 101

Student B $\rightarrow$ id = 102

Then:

A.compareTo(B)

produces a negative result:

$101 < 102$

Therefore:

A comes before B

Conversely:

B.compareTo(A)

produces a positive result:

$102 > 101$

And:

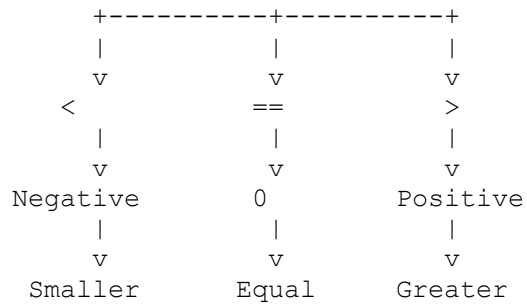
A.compareTo(A)

produces:

0

Conceptually:

```
compareTo()  
  |  
  v  
Compare this.id with other.id  
  |
```



6. Comparable with TreeSet

Comparable is particularly important with sorted collections such as TreeSet.

```
import java.util.TreeSet;

class Employee implements Comparable<Employee> {

    private int id;
    private String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class ComparableTreeSetExample {

    public static void main(String[] args) {

        TreeSet<Employee> employees = new TreeSet<>();

        employees.add(new Employee(103, "Amit"));
        employees.add(new Employee(101, "Anurag"));
        employees.add(new Employee(102, "Rahul"));

        System.out.println(employees);
    }
}
```

Output:

[101 - Anurag, 102 - Rahul, 103 - Amit]

The TreeSet uses compareTo() to determine the ordering.

7. Comparable and Duplicate Detection

An important concept is that sorted collections such as TreeSet use the comparison result to determine whether two objects are equivalent **for ordering purposes**.

For example:

```
@Override
public int compareTo(Employee other) {
    return Integer.compare(this.id, other.id);
}
```

If:

Employee A → id = 101

Employee B → id = 101

then:

A.compareTo(B)

returns:

0

Therefore, a TreeSet can treat them as the same element for set ordering.

```
Employee A (101)
    |
    | compareTo()
    v
Employee B (101)
    |
    v
    0
    |
    v
TreeSet considers them equivalent
for its ordering
```

This is why the field used by compareTo() should be chosen carefully.

8. Comparable with Collections.sort()

```
List<Integer> numbers = new ArrayList<>();
```

```
numbers.add(50);  
numbers.add(10);  
numbers.add(30);  
numbers.add(20);
```

```
Collections.sort(numbers);
```

```
System.out.println(numbers);
```

Output:

```
[10, 20, 30, 50]
```

Integer already implements Comparable<Integer>, so Java knows its natural ordering.

Conceptually:

```
Integer  
  |  
  v  
implements Comparable<Integer>  
  |  
  v  
compareTo()  
  |  
  v  
Natural numeric ordering
```

9. Comparable with Strings

String also has a natural ordering.

```
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.List;  
  
public class ComparableStringExample {  
    public static void main(String[] args) {  
        List<String> languages = new ArrayList<>();  
        languages.add("Python");  
        languages.add("Java");  
    }  
}
```

```
        languages.add("C++");  
  
        Collections.sort(languages);  
  
        System.out.println(languages);  
    }  
}
```

Output:

[C++, Java, Python]

The natural ordering of String is lexicographic.

10. Important Comparable Method

Comparable primarily defines one method:

```
int compareTo(T o);
```

Result	Meaning
< 0	Current object comes before o
0	Current object and o are equivalent for ordering
> 0	Current object comes after o

Example:

```
@Override  
public int compareTo(Student other) {  
  
    if (this.id < other.id) {  
        return -1;  
    }  
  
    if (this.id > other.id) {  
        return 1;  
    }  
  
    return 0;  
}
```

A more concise modern version is:

```
@Override
public int compareTo(Student other) {
    return Integer.compare(this.id, other.id);
}
```

11. Comparable vs Comparator

This is one of the most important distinctions.

Feature	Comparable	Comparator
Package	java.lang	java.util
Defines	Natural ordering	External/custom ordering
Main method	compareTo()	compare()
Implemented by	The class being compared	Separate class/object/lambda
Number of orderings	Usually one natural ordering	Multiple possible orderings
Example	Sort Employee by ID	Sort Employee by name
Java 1.2	Yes	Yes

Conceptually:

Comparable

```
Employee
|
+--> implements Comparable
|
+--> compareTo()
|
v
Natural ordering
```

Whereas:

Comparator

```
Employee
|
|
v
```

```
Comparator
|
+--> compare()
|
+--> By ID
+--> By Name
+--> By Salary
|
v
Custom ordering
```

12. Example: Comparable by ID

```
class Employee implements Comparable<Employee> {

    int id;
    String name;

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }
}
```

Natural ordering:

```
101 → Anurag
102 → Rahul
103 → Amit
```

13. Example: Comparator by Name

If you want a different ordering without changing the natural ordering:

```
import java.util.Comparator;

Comparator<Employee> byName =
    Comparator.comparing(employee -> employee.name);
```

Now the same Employee objects can be sorted by name.

This demonstrates the main philosophy:

```
Comparable
↓
"What is the object's natural order?"
```

Comparator



"How do I want to order these objects this time?"

14. Comparable Contract

A good `compareTo()` implementation should follow the comparison contract.

Sign relationship

For objects `x` and `y`:

```
x.compareTo(y) < 0
```



`x` comes before `y`

```
x.compareTo(y) == 0
```



`x` and `y` are equivalent for ordering

```
x.compareTo(y) > 0
```



`x` comes after `y`

Important properties

The ordering should generally be:

- **Antisymmetric in sign**
- **Transitive**
- **Consistent**
- Ideally consistent with `equals()`

The Java API specifically recommends that natural ordering be consistent with `equals()` when practical, especially because sorted collections can otherwise behave unexpectedly.

15. Comparable Flow with `Collections.sort()`

```
List<Student>
```



```
Collections.sort()
```



Compare Student objects

```

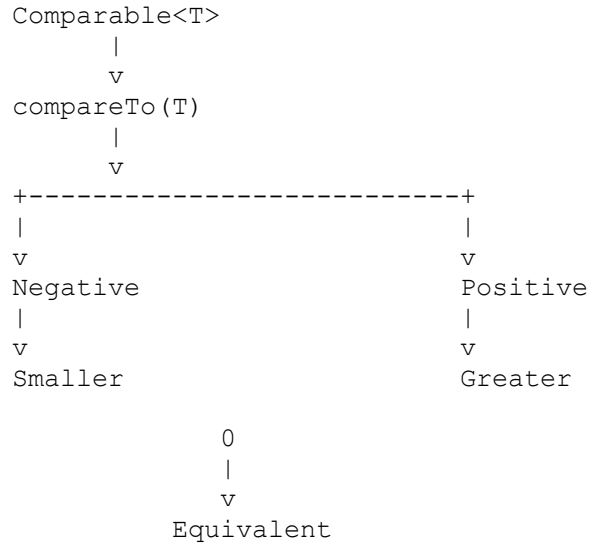
      |
      v
Student.compareTo()
      |
      v
Compare IDs
      |
      v
Determine ordering
      |
      v
Rearrange elements
      |
      v
Sorted List

```

16. Key Characteristics

Characteristic	Comparable
Java Version	1.2
Year	1998
Package	java.lang
Type	Generic interface
Main method	compareTo(T)
Purpose	Define natural ordering
Implemented by	Class whose objects are compared
Used by	Sorting and sorted collections
Works with Collections.sort()	Yes
Works with TreeSet	Yes
Works with TreeMap keys	Yes
Multiple orderings	Not directly; normally one natural ordering
Alternative	Comparator

Key Concept



In one sentence: Comparable is a Java 1.2 interface that allows a class to define its **natural ordering** through the compareTo() method, enabling objects to be sorted and used effectively in sorted collections such as TreeSet and TreeMap.

Java Comparator

Java Version: 1.2

Year: 1998

Feature: Comparator

UniqueClassName: Comparator

1. Definition

Comparator<T> is a **functional interface** in java.util used to define a **custom ordering** for objects.

It allows you to compare two objects **without requiring the class itself to implement Comparable**.

The primary method is:

```
int compare(T o1, T o2)
```

The result means:

Negative value → o1 comes before o2
0 → o1 and o2 are equal for ordering
Positive value → o1 comes after o2

Simple concept

Comparable



Defines object's natural ordering

Comparator



Defines an external/custom ordering

Comparator was introduced in **Java 1.2 (1998)** along with the Collections Framework.

2. Comparator Hierarchy

Comparator is an interface in `java.util`.

`java.util`

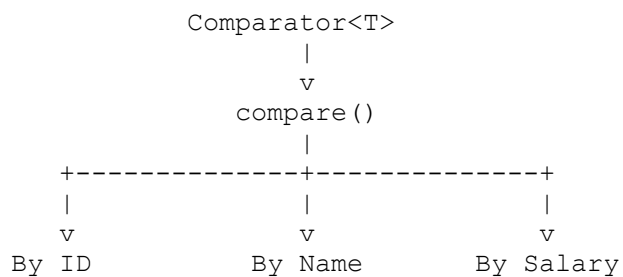


`Comparator<T>`



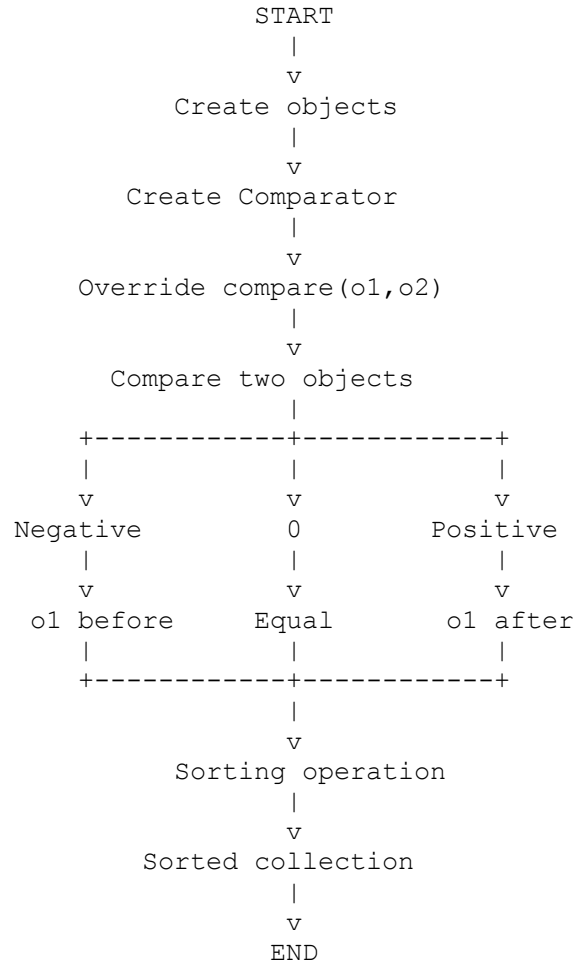
`compare(T o1, T o2)`

Conceptually:



Unlike Comparable, the comparator is normally **separate from the class being compared**.

3. Comparator Flowchart



4. Basic Java Code Example

ComparatorExample.java

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

class Student {

    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
    }
}
```

```

        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class ComparatorExample {

    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        students.add(new Student(103, "Amit"));
        students.add(new Student(101, "Anurag"));
        students.add(new Student(102, "Rahul"));

        System.out.println("Before Sorting:");
        System.out.println(students);

        // Comparator: sort by ID
        Comparator<Student> byId =
            new Comparator<Student>() {

                @Override
                public int compare(Student s1, Student s2) {
                    return Integer.compare(
                        s1.getId(),
                        s2.getId()
                    );
                }
            };

        Collections.sort(students, byId);

        System.out.println("\nAfter Sorting by ID:");
        System.out.println(students);
    }
}

```

Output

```

Before Sorting:
[103 - Amit, 101 - Anurag, 102 - Rahul]

```

After Sorting by ID:
[101 - Anurag, 102 - Rahul, 103 - Amit]

5. How compare() Works

Suppose:

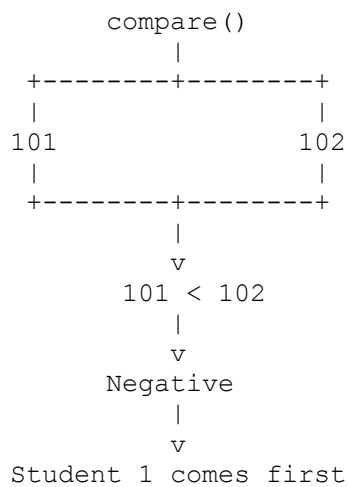
Student 1 → ID 101

Student 2 → ID 102

The comparator executes:

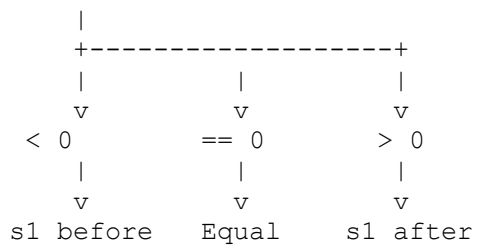
`compare(student1, student2)`

Conceptually:



The three possible results are:

`compare(s1, s2)`



6. Comparator by Name

The same Student class can be sorted by name without modifying the class.

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class ComparatorNameExample {

    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        students.add(new Student(103, "Rahul"));
        students.add(new Student(101, "Amit"));
        students.add(new Student(102, "Anurag"));

        Comparator<Student> byName =
            new Comparator<Student>() {

                @Override
                public int compare(Student s1, Student s2) {
                    return s1.getName()
                        .compareTo(s2.getName());
                }

            };

        students.sort(byName);

        System.out.println(students);
    }
}
```

Output:

```
[101 - Amit, 102 - Anurag, 103 - Rahul]
```

Here:

```
Student
|
+--> ID
|
+--> Name
|
v
Comparator
|
v
Sort by Name
```

7. Comparator by Descending ID

A comparator can also define descending order.

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class ComparatorDescendingExample {

    public static void main(String[] args) {

        List<Student> students = new ArrayList<>();

        students.add(new Student(101, "Anurag"));
        students.add(new Student(103, "Amit"));
        students.add(new Student(102, "Rahul"));

        Comparator<Student> descendingId =
            new Comparator<Student>() {

                @Override
                public int compare(Student s1, Student s2) {
                    return Integer.compare(
                        s2.getId(),
                        s1.getId()
                    );
                }
            };

        students.sort(descendingId);

        System.out.println(students);
    }
}
```

Output:

```
[103 - Amit, 102 - Rahul, 101 - Anurag]
```

8. Multiple Comparators

One of the biggest advantages of Comparator is that a class can have **multiple sorting strategies**.

For example:

```

          Student
            |
+-----+-----+
|       |       |
v       v       v
```

By ID	By Name	By another field
v	v	v
Comparator	Comparator	Comparator

Example:

```
Comparator<Student> byId =
    Comparator.comparingInt(Student::getId);
```

```
Comparator<Student> byName =
    Comparator.comparing(Student::getName);
```

Then:

```
students.sort(byId);
```

or:

```
students.sort(byName);
```

This capability became especially concise with **lambda expressions and Comparator utility methods in Java 8**.

9. Comparator with TreeSet

Comparator is particularly useful when creating a TreeSet with custom ordering.

```
import java.util.Comparator;
import java.util.TreeSet;

public class ComparatorTreeSetExample {

    public static void main(String[] args) {

        TreeSet<Student> students =
            new TreeSet<>(
                Comparator.comparingInt(Student::getId)
            );

        students.add(new Student(103, "Amit"));
        students.add(new Student(101, "Anurag"));
        students.add(new Student(102, "Rahul"));

        System.out.println(students);
    }
}
```

Output:

```
[101 - Anurag, 102 - Rahul, 103 - Amit]
```

Conceptually:

```
TreeSet
|
v
Comparator
|
v
Compare Student IDs
|
v
Maintain sorted tree
```

10. Comparator with TreeMap

A Comparator can also determine the ordering of keys in a TreeMap.

```
import java.util.Comparator;
import java.util.TreeMap;

public class ComparatorTreeMapExample {

    public static void main(String[] args) {

        TreeMap<Integer, String> map =
            new TreeMap<>(Comparator.reverseOrder());

        map.put(101, "Anurag");
        map.put(103, "Amit");
        map.put(102, "Rahul");

        System.out.println(map);
    }
}
```

Output:

```
{103=Amit, 102=Rahul, 101=Anurag}
```

The comparator causes the keys to be ordered in descending order.

11. Java 8 Lambda Example

Although Comparator was introduced in **Java 1.2**, Java 8 made its use much shorter with lambda expressions.

```
students.sort(  
    (s1, s2) -> Integer.compare(  
        s1.getId(),  
        s2.getId()  
    )  
);
```

Or:

```
students.sort(  
    Comparator.comparingInt(Student::getId)  
);
```

Conceptual evolution

Java 1.2
|
v
Anonymous Comparator class
|
v
Java 8
|
v
Lambda expression
|
v
Comparator utility methods

12. Comparator.comparing()

Modern Java provides convenient factory methods.

```
Comparator<Student> byName =  
    Comparator.comparing(Student::getName);
```

For an integer:

```
Comparator<Student> byId =  
    Comparator.comparingInt(Student::getId);
```

For descending order:

```
Comparator<Student> byIdDescending =  
    Comparator.comparingInt(Student::getId)  
                .reversed();
```

13. Multiple-Level Sorting

A Comparator can define a primary and secondary ordering.

For example:

```
Comparator<Student> comparator =  
    Comparator.comparing(Student::getName)  
                .thenComparingInt(Student::getId);
```

Meaning:

First sort by Name

|

v

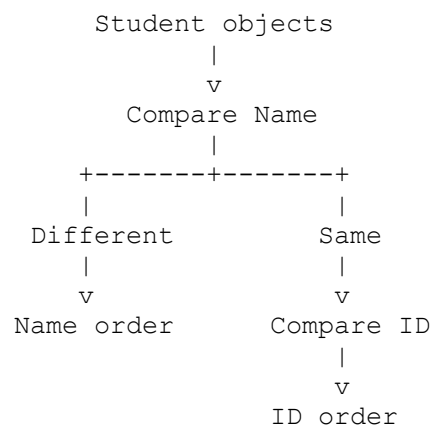
If names are equal

|

v

Sort by ID

Flowchart:



14. Comparable vs Comparator

This is the most important distinction.

Feature	Comparable	Comparator
Package	java.lang	java.util
Java version	1.2	1.2
Main method	compareTo()	compare()
Defined by	The class itself	Separate comparator
Purpose	Natural ordering	Custom ordering
Modify original class?	Usually yes	No
Multiple sorting strategies	Not directly	Yes
Example	Employee natural order by ID	Employee by name/salary
Used by TreeSet	Yes	Yes
Used by TreeMap	Yes	Yes
Lambda support	No direct lambda for Comparable itself	Yes, Java 8+

Simple comparison

Comparable

|

v

Class itself defines ordering

Employee

|

+--> implements Comparable<Employee>

|

+--> compareTo()

|

v

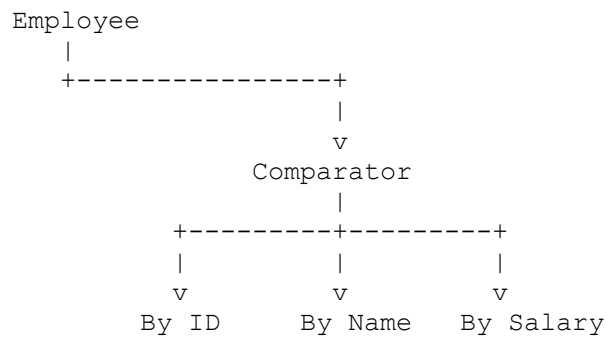
Natural ordering

Comparator

|

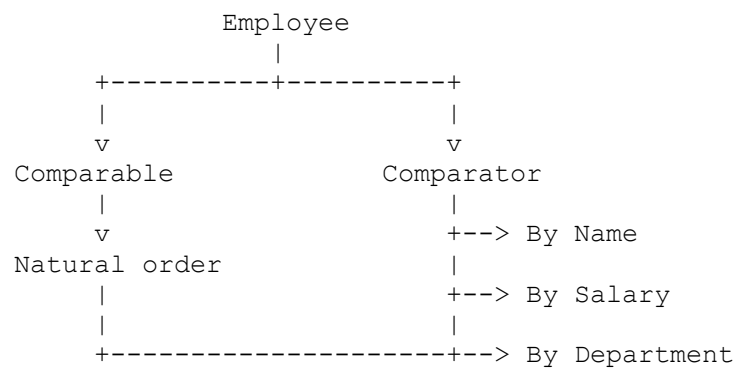
v

External object defines ordering



15. Comparable + Comparator Together

A class can have a natural ordering using Comparable while also supporting additional orderings through Comparator.



Example:

```

class Employee implements Comparable<Employee> {
    private int id;

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.id, other.id);
    }
}
  
```

Natural order:

Employee → ID

Additional ordering:

```
Comparator<Employee> byName =  
    Comparator.comparing(Employee::getName);
```

Custom order:

Employee → Name

16. Important Comparator Methods

The original Java 1.2 Comparator contract centered on:

```
int compare(T o1, T o2);
```

Modern Java has many additional default/static methods.

Method	Purpose
compare()	Compares two objects
reversed()	Reverses ordering
thenComparing()	Adds secondary ordering
comparing()	Creates comparator from a key
comparingInt()	Creates comparator for integer key
comparingLong()	Creates comparator for long key
comparingDouble()	Creates comparator for double key
naturalOrder()	Natural-order comparator
reverseOrder()	Reverse natural ordering
nullsFirst()	Places null values first
nullsLast()	Places null values last

Many of these utility methods were added in later Java releases, particularly Java 8.

17. Comparator Contract

For:

`compare(o1, o2)`

the result should follow:

`< 0`

↓

`o1 comes before o2`

`= 0`

↓

`o1 and o2 are equivalent for ordering`

`> 0`

↓

`o1 comes after o2`

The comparison should generally be:

- **antisymmetric in sign**
- **transitive**
- consistent with repeated comparisons

For sorted collections, a comparator that is inconsistent with `equals()` can produce surprising behavior, because sorted collections use the comparison result to determine ordering/equivalence.

18. Key Characteristics

Characteristic	Comparator
Java Version	1.2
Year	1998
Package	java.util
Type	Interface
Main method	<code>compare(T o1, T o2)</code>
Purpose	Custom object ordering
Defines natural ordering?	No

External ordering	Yes
Multiple orderings	Yes
Works with Collections.sort()	Yes
Works with List.sort()	Yes
Works with TreeSet	Yes
Works with TreeMap	Yes
Lambda support	Yes, since Java 8
Utility methods	Expanded significantly in Java 8

Key Concept

```

Comparator<T>
  |
  v
compare(o1, o2)
  |
  +-----+
  |         |         |
  v         v         v
< 0        == 0       > 0
  |         |         |
  v         v         v
o1 before  Equal    o1 after

```

Final Difference

```

Comparable
  ↓
"How should objects of THIS class naturally be ordered?"
  ↓
compareTo()

```

```

Comparator
  ↓
"How should I order THESE objects for this particular purpose?"
  ↓
compare()

```

In one sentence: Comparator is a Java 1.2 interface that provides an **external/custom ordering strategy** for objects through `compare()`, allowing the same class to be sorted in multiple different ways.

Java Swing UI

Java Version: 1.2

Year: 1998

Feature: Swing UI

UniqueClassName: SwingIntro

1. Definition

Swing is a Java GUI (Graphical User Interface) toolkit used to build **desktop applications** with windows, buttons, text fields, menus, tables, trees, dialogs, and other graphical components.

Swing was introduced in the **Java 1.2 era (1998)** as part of the **Java Foundation Classes (JFC)**.

Swing components are primarily provided by the:

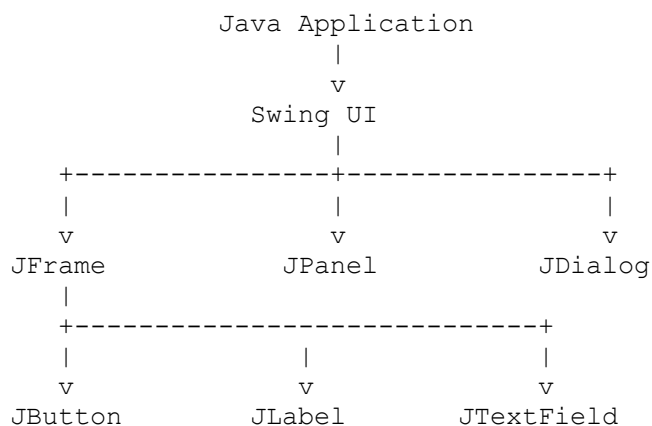
`javax.swing`

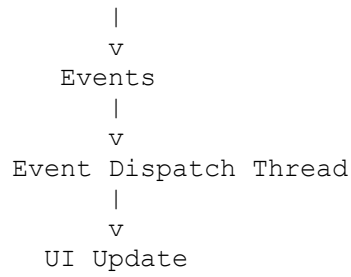
package.

Examples include:

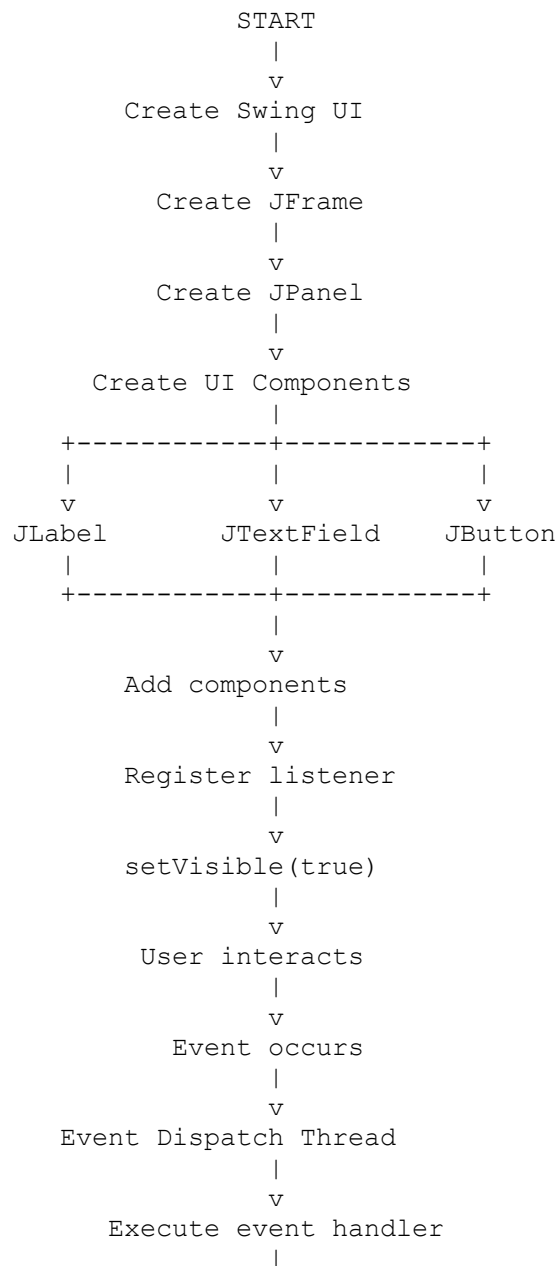
JFrame	→ Main application window
JPanel	→ Container
JButton	→ Button
JLabel	→ Text/image label
JTextField	→ Single-line input
JTextArea	→ Multi-line text
JCheckBox	→ Checkbox
JRadioButton	→ Radio button
JComboBox	→ Drop-down list
JTable	→ Table
JTree	→ Tree
JDialog	→ Dialog window

2. Swing Architecture





3. Swing Flowchart



```
      v
Update UI
      |
      v
    END
```

4. Basic Swing Java Code Example

SwingIntro.java

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import java.awt.FlowLayout;

public class SwingIntro {

    public static void main(String[] args) {

        // Create the main window
        JFrame frame = new JFrame("Swing Application");

        // Create panel
        JPanel panel = new JPanel();

        // Create components
        JLabel label = new JLabel("Enter your name:");

        JTextField textField =
            new JTextField(15);

        JButton button =
            new JButton("Submit");

        // Add components to panel
        panel.add(label);
        panel.add(textField);
        panel.add(button);

        // Add panel to frame
        frame.add(panel);

        // Set layout
        panel.setLayout(new FlowLayout());

        // Configure frame
        frame.setSize(400, 150);
        frame.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE
        );
    }
}
```

```

        // Display window
        frame.setVisible(true);
    }
}

```

Conceptual UI

```

+-----+
|           Swing Application           |
+-----+
| Enter your name: [ _____ ]       |
|                                     |
|           [ Submit ]                 |
|                                     |
+-----+

```

5. Swing with Button Event

A GUI becomes interactive by handling events.

```

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import java.awt.FlowLayout;

public class SwingIntro {

    public static void main(String[] args) {

        JFrame frame =
            new JFrame("Swing Example");

        JPanel panel =
            new JPanel(new FlowLayout());

        JLabel label =
            new JLabel("Name:");

        JTextField textField =
            new JTextField(15);

        JButton button =
            new JButton("Submit");

        button.addActionListener(event -> {

            String name = textField.getText();

            label.setText("Hello, " + name);

```

```

        });

        panel.add(label);
        panel.add(textField);
        panel.add(button);

        frame.add(panel);

        frame.setSize(400, 150);

        frame.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE
        );

        frame.setVisible(true);
    }
}

```

Flow

```

User
|
| Enters name
v
JTextField
|
| Clicks Submit
v
JButton
|
v
ActionListener
|
v
Read getText()
|
v
Update JLabel
|
v
"Hello, Anurag"

```

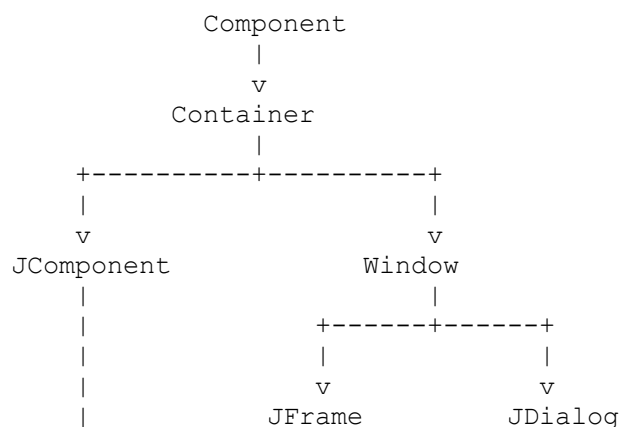
The lambda expression used above requires Java 8+. The Swing framework itself dates to the Java 1.2 era.

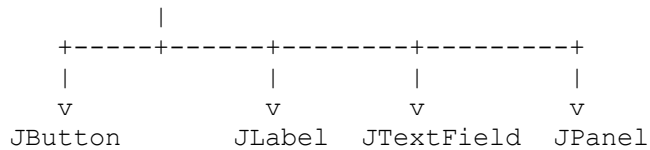
6. Important Swing Components

Component	Purpose	Example
JFrame	Main window	Application window
JPanel	Container	Groups components
JLabel	Displays text/image	"Name:"
JButton	Clickable button	"Submit"
TextField	Single-line input	Name
TextArea	Multi-line input	Description
CheckBox	Boolean selection	"Java"
RadioButton	Single choice	"Male" / "Female"
ComboBox	Drop-down selection	Country
JList	List selection	Languages
JTable	Tabular data	Employee table
JTree	Hierarchical data	File tree
JMenuBar	Menu bar	File/Edit
JDialog	Dialog window	Confirmation

7. Swing Component Hierarchy

A simplified hierarchy looks like:

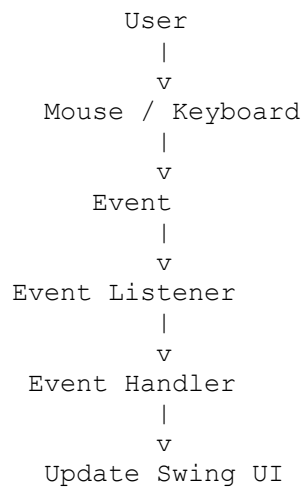




JFrame and JDialog are top-level containers rather than subclasses of JComponent.

8. Swing Event Handling

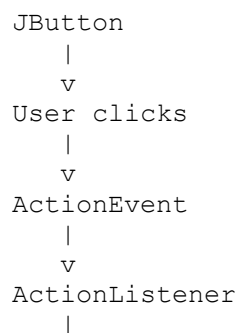
Swing follows an **event-driven programming model**.



For a button:

```
button.addActionListener(event -> {
    System.out.println("Button clicked");
});
```

The relationship is:



v
Handler executes

9. Swing and Event Dispatch Thread

Swing uses the **Event Dispatch Thread (EDT)** for processing GUI events and updating Swing components.

A recommended application startup pattern is:

```
import javax.swing.JFrame;
import javax.swing.SwingUtilities;

public class SwingIntro {

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {

            JFrame frame =
                new JFrame("Swing Application");

            frame.setSize(400, 200);

            frame.setDefaultCloseOperation(
                JFrame.EXIT_ON_CLOSE
            );

            frame.setVisible(true);

        });

    }

}
```

Flow:

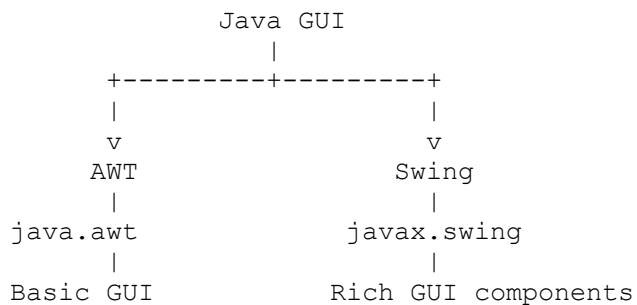
```
main()
|
v
SwingUtilities.invokeLater()
|
v
Event Dispatch Thread
|
v
Create Swing UI
|
v
Display JFrame
```

10. Swing vs AWT

Swing was built on top of the earlier **AWT (Abstract Window Toolkit)**.

Feature	AWT	Swing
Introduced	Java 1.0	Java 1.2 era
Package	java.awt	javax.swing
Components	Mostly heavyweight	Mostly lightweight
Look and Feel	Platform dependent	Pluggable Look and Feel
Rich components	Limited	Extensive
JTable	No	Yes
JTree	No	Yes
JTabbedPane	No	Yes
JFrame	No	Yes

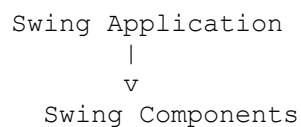
Conceptually:

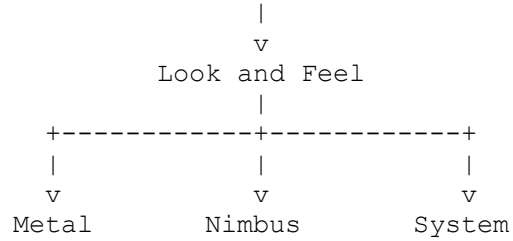


11. Swing's Pluggable Look and Feel

One important Swing feature is its **pluggable Look and Feel**.

Conceptually:





The same application can use different visual styles without changing its core application logic.

12. Simple Login UI Example

```

import javax.swing.*;

public class SwingLoginExample {

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {

            JFrame frame =
                new JFrame("Login");

            JPanel panel =
                new JPanel();

            JLabel userLabel =
                new JLabel("Username:");

            JTextField username =
                new JTextField(15);

            JLabel passwordLabel =
                new JLabel("Password:");

            JPasswordField password =
                new JPasswordField(15);

            JButton login =
                new JButton("Login");

            login.addActionListener(event -> {

                String user =
                    username.getText();

                JOptionPane.showMessageDialog(
                    frame,
                    "Welcome " + user
                );
            });
        });
    }
}

```

```

        panel.add(userLabel);
        panel.add(username);

        panel.add(passwordLabel);
        panel.add(password);

        panel.add(login);

        frame.add(panel);

        frame.setSize(350, 200);

        frame.setDefaultCloseOperation(
            JFrame.EXIT_ON_CLOSE
        );

        frame.setVisible(true);
    });
}

```

Conceptual UI:

```

+-----+
|               Login               |
+-----+
| Username:  [ _____ ]         |
| Password:  [ _____ ]         |
|                                     |
|               [ Login ]           |
+-----+

```

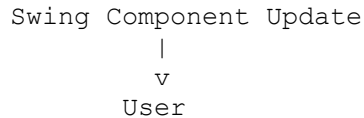
13. Swing Architecture

A simplified Swing application architecture can be represented as:

```

      User
      |
      v
    Swing Component
      |
      v
      Event
      |
      v
Event Dispatch Thread
      |
      v
    Application Logic
      |
      v

```



Swing also follows the broader **Model-View-Controller (MVC)** design principles for many components, although Swing's implementation often uses a modified/separated model architecture rather than a textbook MVC implementation.

14. Key Characteristics of Swing

Characteristic	Swing
Java Version	1.2 era
Year	1998
Main package	javax.swing
Purpose	Desktop GUI applications
GUI model	Event-driven
Main window	JFrame
Container	JPanel
Button	JButton
Text input	JTextField
Rich components	Yes
Tables	JTable
Trees	JTree
Dialogs	JDialog, JOptionPane
Custom Look & Feel	Yes
Event Dispatch Thread	Yes
Platform independent	Yes, at Java application level

Key Concept

```
Swing
|
+--> Desktop GUI
|
+--> javax.swing
|
+--> JFrame
+--> JPanel
+--> JButton
+--> JLabel
+--> JTextField
+--> JTable
+--> JTree
+--> JDialog
|
+--> Event-driven
|
+--> Event Dispatch Thread
|
+--> Pluggable Look & Feel
```

In one sentence: **Swing** is Java's rich desktop GUI toolkit introduced in the **Java 1.2 era (1998)** as part of the Java Foundation Classes, providing components such as JFrame, JButton, JLabel, JTextField, JTable, and JTree for building interactive desktop applications.

Java Security Permissions / Java 2 Security Model

Java Version: 1.2

Year: 1998

Feature: Security permissions

UniqueClassName: SecurityModel

1. Definition

The **Java 2 Security Model**, introduced with Java 1.2, provided a configurable **permission-based access-control mechanism** for protecting sensitive resources such as files, network connections, system properties, and runtime operations.

A permission represents authorization to perform a particular operation on a particular resource. Java associates loaded code with a **protection domain**, and the security policy determines which permissions that code receives. ([Oracle Documentation](#))

The core concepts were:

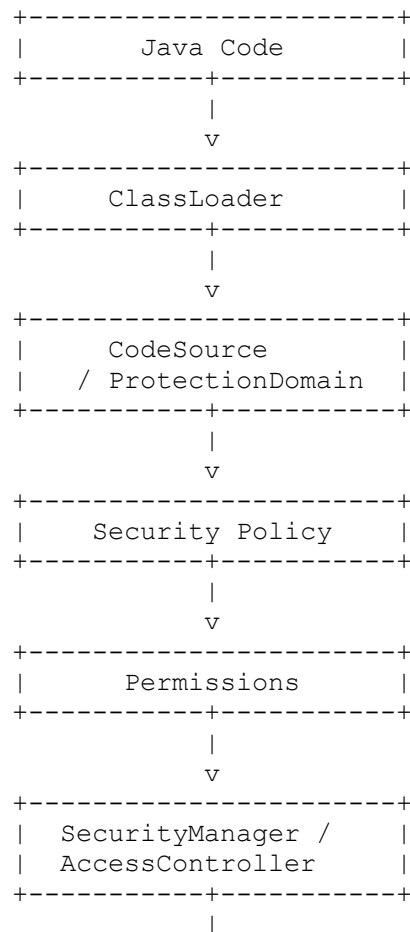
```

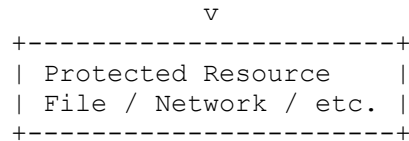
Code
|
v
ClassLoader
|
v
CodeSource / ProtectionDomain
|
v
Security Policy
|
v
Permissions
|
v
Access Control

```

For example, a policy could grant code permission to read a particular file but deny it permission to write that file.

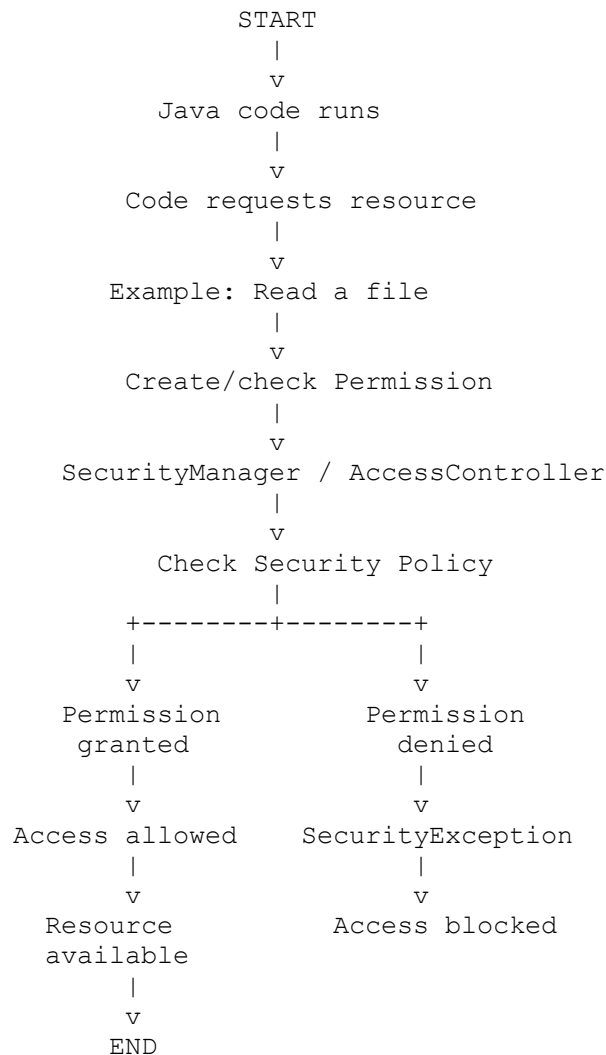
2. Main Security Components





The Java security architecture uses `SecurityManager` as a central access-control mechanism and `AccessController` to perform permission checks according to the configured policy.

3. Security Permission Flowchart



Java's access-control mechanism can evaluate the call stack against the permissions granted by the policy; if the required permission is not granted, a security exception is raised. ([Oracle Documentation](#))

4. Important Permission Classes

Java provides different permission types for different resources.

Permission	Purpose	Example
FilePermission	File/directory access	Read/write a file
SocketPermission	Network access	Connect to a host
RuntimePermission	Runtime operations	Certain JVM operations
PropertyPermission	System properties	Read/write properties
AllPermission	Grants all permissions	Highly privileged code

For example:

```
FilePermission permission =  
    new FilePermission("data.txt", "read");
```

This represents permission to read data.txt. ([Oracle Documentation](#))

5. Basic Java Code Example

SecurityModel.java

```
import java.io.FilePermission;  
import java.security.AccessController;  
  
public class SecurityModel {  
    public static void main(String[] args) {  
        FilePermission permission =  
            new FilePermission(  
                "data.txt",  
                "read"  
            );  
  
        try {  
            AccessController.checkPermission(  
                permission  
            );  
        }  
    }  
}
```

```

        System.out.println(
            "Permission granted."
        );
    } catch (SecurityException e) {
        System.out.println(
            "Permission denied."
        );
    }
}

```

Conceptual Flow

```

FilePermission
|
v
"data.txt" + "read"
|
v
AccessController.checkPermission()
|
v
Security Policy
|
+---+---+
|       |
YES     NO
|       |
v       v
Allow   SecurityException

```

Important: Whether this produces "granted" or "denied" depends on the security configuration/runtime in which the program is executed. Modern JDKs no longer support the old Security Manager mechanism; this example is primarily useful for understanding the Java 1.2 security model. ([Oracle Documentation](#))

6. File Permission Example

A classic Java 2 security scenario is controlling file access.

```

import java.io.FilePermission;

public class SecurityModel {

    public static void main(String[] args) {

        FilePermission readPermission =
            new FilePermission(

```

```

        "/tmp/data.txt",
        "read"
    );

    FilePermission writePermission =
        new FilePermission(
            "/tmp/data.txt",
            "write"
        );

    System.out.println(
        "Read permission: "
        + readPermission
    );

    System.out.println(
        "Write permission: "
        + writePermission
    );
}
}

```

Conceptually:

```

Application
|
+---- read /tmp/data.txt
|       |
|       v
|       FilePermission
|
+---- write /tmp/data.txt
|       |
|       v
|       FilePermission

```

The two operations can be granted independently.

7. Security Policy Concept

A Java security policy defines which code is allowed to perform which operations.

A simplified policy concept looks like:

```

Code Source
|
v
Permission Set
|
+--> FilePermission → read

```

```

|
+--> SocketPermission → connect
|
+--> PropertyPermission → read

```

A historical Java policy entry could look conceptually like:

```

grant {
  permission java.io.FilePermission
    "/tmp/data.txt", "read";
};

```

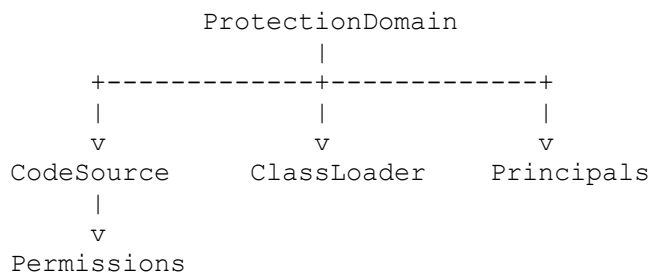
This means the corresponding code domain is granted permission to read that resource.

The policy mechanism was part of the Java 2 security architecture and was represented by `java.security.Policy`. ([Oracle Documentation](#))

8. Protection Domain

A **ProtectionDomain** associates code with its security characteristics and permissions.

Conceptually:



For example:

```

Application A
|
v
ProtectionDomain A
|
+--> File read
+--> Network connection

```

```

Application B
|
v
ProtectionDomain B
|

```

+--> File read only

Classes are associated with protection domains when loaded, and the permissions associated with those domains are determined by the security policy. ([Oracle Documentation](#))

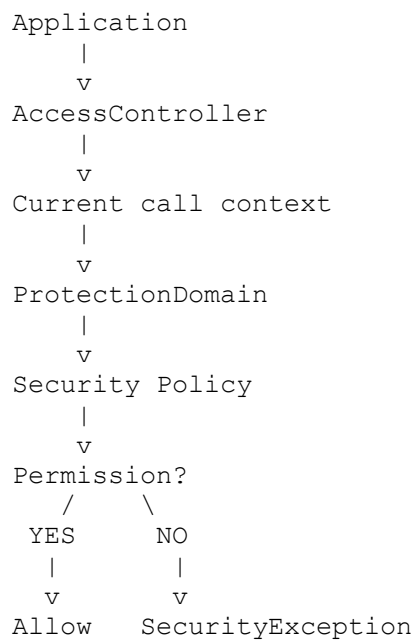
9. AccessController

AccessController is responsible for performing permission decisions according to the Java access-control model.

Basic example:

```
AccessController.checkPermission(permission);
```

Conceptually:



Oracle's documentation describes AccessController as responsible for deciding whether access to a critical system resource should be allowed or denied based on the active security policy. ([Oracle Documentation](#))

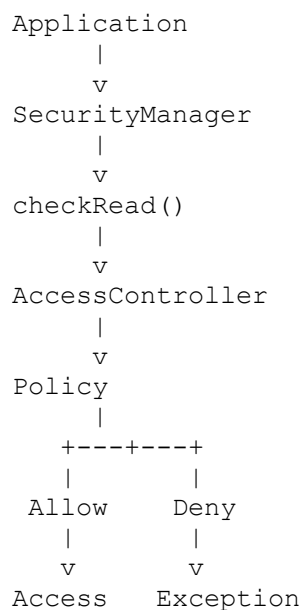
10. SecurityManager

Historically, SecurityManager provided the central point through which security-sensitive operations were checked.

Example:

```
SecurityManager securityManager =  
    System.getSecurityManager();  
  
if (securityManager != null) {  
    securityManager.checkRead(  
        "data.txt"  
    );  
}
```

Flow:



The traditional SecurityManager implementation delegated permission checks to AccessController. ([Oracle Documentation](#))

Modern Java note: SecurityManager and the associated policy mechanism have been deprecated for removal, and current Java releases no longer support the old model as a general application security boundary. ([Oracle Documentation](#))

11. Permission-Based Security Example

Consider an application trying to perform three operations:

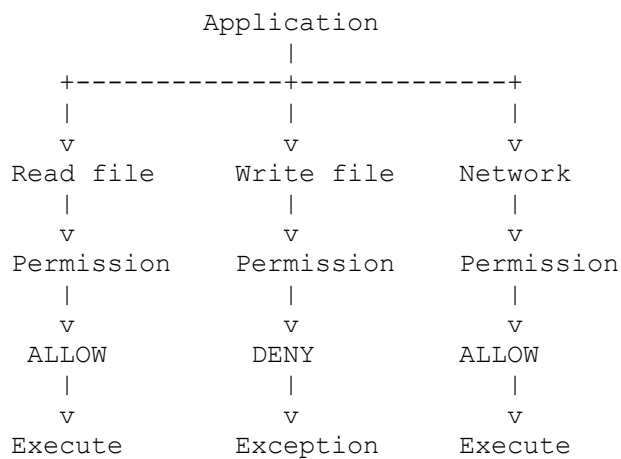
Application

```
|
+---- Read file
|
+---- Write file
|
+---- Connect network
```

The policy might grant:

```
Read file      → ALLOWED
Write file     → DENIED
Network connect → ALLOWED
```

Flowchart:



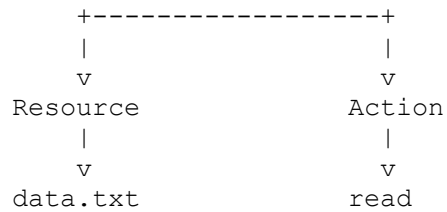
This is the fundamental idea of **least privilege**: code receives only the permissions it needs.

12. Permission Concept

`java.security.Permission` is the abstraction representing access to a resource and, where applicable, an action on that resource. Concrete permissions such as `FilePermission` extend this model. ([Oracle Documentation](#))

Conceptually:

```
Permission
|
```



Example:

```

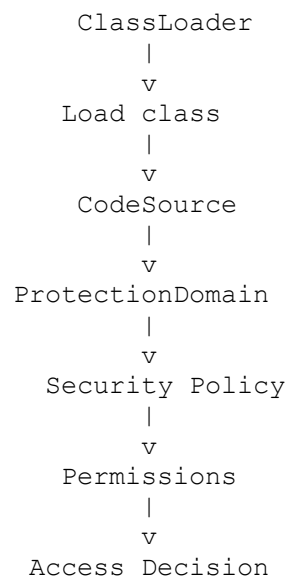
new FilePermission(
    "data.txt",
    "read"
);
  
```

Means:

Target = data.txt
Action = read

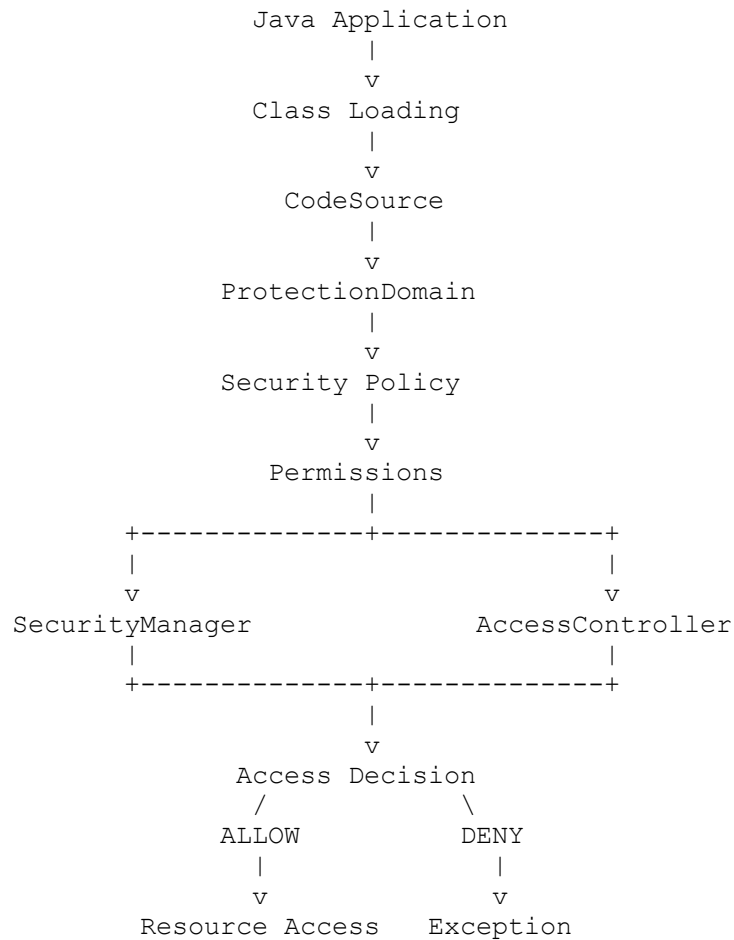
13. Security Model with ClassLoader

Class loading was an important part of the Java 2 security architecture.



The security architecture associates loaded code with information such as its origin/code source, signer information, and permissions. ([Oracle Documentation](#))

14. Java 1.2 Security Model Architecture



This architecture is the basis of the Java 2 permission model. ([Oracle Documentation](#))

15. Important Security APIs

API/Class	Purpose
java.security.Permission	Base permission abstraction
java.io.FilePermission	File/directory access
java.net.SocketPermission	Network access
java.lang.RuntimePermission	Runtime operations
java.util.PropertyPermission	System-property access

java.security.ProtectionDomain	Associates code with security characteristics
java.security.Policy	Determines permissions under the policy model
java.security.AccessController	Performs access-control decisions
java.lang.SecurityManager	Historical central security-check mechanism

16. Key Characteristics

Characteristic	Java 1.2 Security Model
Introduced	Java 1.2
Year	1998
Model	Permission-based access control
Main purpose	Protect sensitive resources
Policy	Configurable
CodeSource	Identifies origin of code
ProtectionDomain	Associates code with permissions
Permission	Represents allowed operation
AccessController	Performs access checks
SecurityManager	Central security-check mechanism
Typical resources	Files, network, system properties, runtime
Principle	Least privilege

Key Concept

```

Java 1.2 Security Model
|
+--> CodeSource
|
+--> ProtectionDomain

```

```
|
+--> Security Policy
|
+--> Permissions
|
+--> AccessController
|
+--> SecurityManager
|
v
Allow or Deny
```

In one sentence: The **Java 1.2 Security Model** introduced a configurable permission-based architecture in which code was associated with protection domains and granted specific permissions, allowing security-sensitive operations to be **allowed or denied according to policy**. ([Oracle Documentation](#))